Localisation de tumeurs cérébrales

Data Science – Ruben Barata, Arthur De Rouck, Marie Hoizey-Cangioloni, Pedro Lubaszewski Lima, Mohammad Seyedi

CentraleSupélec
June 6, 2026

Disponibilité du code. L'ensemble du code source et des scripts sont disponibles publiquement à l'adresse suivante :
https://github.com/PLLima/Pediatric-Brain-Tumor-MultiOmics

Contents

1 Introduction 3

2 Analyse exploratoire des données (EDA) 5
- 2.1 Structure de la cohorte : $p \ggg n$, classe rare et bloc dominant . . . 5
- 2.2 Qualité des données et prétraitement . . . 6
- 2.3 Séparabilité non supervisée : PCA par bloc . . . 7
- 2.4 Structure de corrélation et redondance . . . 8
- 2.5 Signal univarié autour de `midl` . . . 9
- 2.6 Choix de la métrique d'évaluation . . . 10
- 2.7 Implications pour la modélisation . . . 10

3 Formalisme mathématique 11
- 3.1 Notion d'entropie croisée[1] . . . 11
- 3.2 Modélisation du problème[2] . . . 11
- 3.3 Régression logistique multiclasse[3] . . . 12
  - 3.3.1 Vraisemblance pondérée et choix des poids de classe . . . 13
- 3.4 Minimisation du risque empirique pénalisé : Elastic-Net[4] . . . 15
- 3.5 GLM[5] . . . 15
- 3.6 Linear Discriminant Analysis (LDA) . . . 18
- 3.7 Cross-Validation . . . 20
- 3.8 Méthodologie de fusion de données multiblocs . . . 21
  - 3.8.1 Méthodes naïves . . . 21
  - 3.8.2 Cooperative Learning . . . 21
- 3.9 RGCCA et SGCCA . . . 23

4 Résultats 26
- 4.1 Sélection de variables . . . 26

5 Stabilité de nos résultats 33
- 5.1 Stabilité de la sélection de variables . . . 33
- 5.2 Reproductibilité et synthèse . . . 34

[1] Cette section s'appuie sur le cours de théorie de l'information, CentraleSupélec.
[2] Cette section s'appuie sur le cours de machine learning théorique, ENSAE.
[3] Cette section s'appuie sur les cours de machine learning théorique et d'optimisation, ENSAE.
[4] Cette section s'appuie sur le cours de statistiques, CentraleSupélec.
[5] Cette section s'appuie sur les cours de Big Data & Santé (CentraleSupélec) et de machine learning théorique (ENSAE).

1 Introduction

Les tumeurs cérébrales constituent la première cause de mortalité par cancer chez l'enfant, et leur prise en charge dépend étroitement de leur localisation anatomique : à histologie comparable, une tumeur du tronc cérébral et une tumeur corticale n'appellent ni la même résécabilité chirurgicale, ni le même protocole de radiothérapie, ni le même pronostic. Déterminer la localisation à partir de la seule signature moléculaire de la tumeur présente donc un intérêt clinique direct, tant pour orienter la stratégie thérapeutique que pour comprendre les mécanismes biologiques propres à chaque site.

Notre projet porte sur la classification de la localisation de tumeurs cérébrales pédiatriques à partir de données moléculaires multi-omiques. La classification est opérée selon 3 classes :

- cort (cortical) : ces tumeurs se situent dans le cortex cérébral, c'est-à-dire dans la couche superficielle des hémisphères cérébraux.
- dipg : tumeurs situées spécifiquement dans le pont (pont de Varole), une zone du cerveau agissant comme un centre de contrôle vital. Cette localisation rend impossible toute opération.
- midl (ligne médiane) : tumeurs situées le long de la médiane du cerveau, en dehors du pont.

Ces tumeurs sont très différentes, tant sur le plan biologique qu'anatomique. L'objectif final de ce travail est de guider la stratégie thérapeutique.

Sur le plan statistique, le problème cumule trois difficultés que l'analyse exploratoire (§2) met en évidence et qui structurent l'ensemble de nos choix méthodologiques : un régime de très grande dimension ($p \simeq 17\,000$ variables pour $n = 39$ patients d'entraînement), une intégration de deux blocs hétérogènes et de tailles très inégales (expression génique et CGH), et un fort déséquilibre de classes, la classe midl ne comptant que 8 patients en entraînement. À chacune de ces difficultés nous opposons une réponse explicite : régularisation et sélection parcimonieuse de variables pour la grande dimension ; mise à l'échelle par bloc et projection supervisée (SGCCA) pour l'hétérogénéité des blocs ; et, pour le déséquilibre, une **pondération de la vraisemblance par l'inverse de la prévalence de chaque classe** (dérivée en §3.3.1), assortie d'une évaluation par balanced accuracy plutôt que par accuracy brute. Le présent rapport est organisé comme suit : la section 2 caractérise le jeu de données ; les sections suivantes formalisent le cadre probabiliste, les modèles pénalisés et les méthodes de fusion multibloc (Cooperative Learning, RGCCA/SGCCA) ; la dernière partie présente les résultats expérimentaux et leur validation statistique.

Les données

Les données multi-omiques proviennent de la cohorte IGR et sont organisés en deux blocs :

Bloc GE (Gene Expression)

- 15 702 features
- Niveaux d'expression génique
- Signal discriminant principal

Bloc CGH (Altérations chromosomiques)

- 1 229 features
- Gains/pertes chromosomiques
- Signal faible isolément

	Train	**Test**
cort	15	5
dipg	16	6
midl	8	3
Total	**39**	**14**

Nous sommes dans un régime $p \gg n$ extrême. La sélection de variables et la régularisation sont donc critiques.
Nous avons remarqué également une autre contrainte : la classe midl est sous représentée !

2 Analyse exploratoire des données (EDA)

Avant toute modélisation, nous avons mené une analyse exploratoire systématique afin de caractériser les difficultés intrinsèques du jeu de données et d'orienter les choix méthodologiques. Cette section est volontairement descriptive : les statistiques supervisées (tests univariés, ANOVA) y sont employées comme diagnostics exploratoires et non comme outil de sélection de variables. Toute sélection effective de features est réalisée à l'intérieur des plis de validation croisée (sections Cross-Validation et Résultats) pour éviter tout data leakage.

Quatre constats structurent l'ensemble de notre travail et seront illustrés tour à tour : (i) un régime $p \ggg n$ extrême avec une classe rare `midl` ; (ii) une domination dimensionnelle massive du bloc GE ; (iii) une absence de séparation non supervisée des classes ; (iv) un signal univarié réel mais peu robuste en multivarié, particulièrement pour `midl`.

2.1 Structure de la cohorte : $p \ggg n$, classe rare et bloc dominant

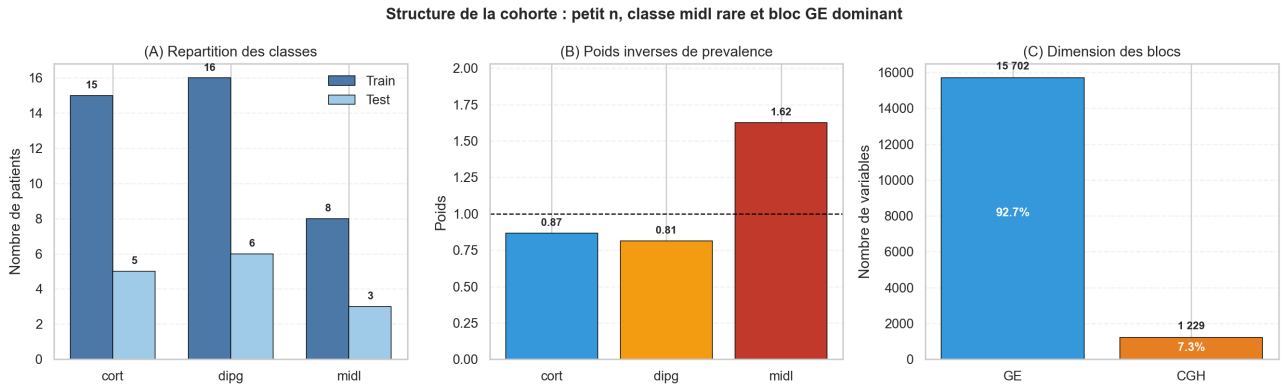


Fig. 1: **(A)** Effectifs par classe en train et test. **(B)** Poids inverses de prévalence $w_c = n/(K\,n_c)$ utilisés pour rééquilibrer la vraisemblance. **(C)** Dimension des deux blocs : GE concentre 92,7% des variables.

Un régime $p \ggg n$ extrême. La cohorte compte $n_{\text{train}} = 39$ patients pour $p = 16\,931$ variables (15 702 GE + 1 229 CGH), soit un rapport

$$\frac{p}{n} = \frac{16\,931}{39} \simeq 434.$$

Dans ce régime, la matrice de Gram empirique $X^\top X$ est de rang au plus $n-1 = 38$: elle est massivement singulière, tout estimateur non régularisé est non identifiable, et le risque de sur-apprentissage est maximal. Ce seul constat justifie le recours systématique à la régularisation (ℓ^1, ℓ^2) et à la sélection parcimonieuse de variables dans toute la suite.

Une classe minoritaire qui constitue le verrou principal. La distribution des classes est déséquilibrée (Fig. 1A) : `cort` (15), `dipg` (16) et `midl` (8) en train. La classe `midl` ne représente que $8/39 \simeq 20{,}5\%$ du train et seulement 3 patients en test. Or chaque erreur sur `midl` déplace son recall par pas de $1/3 \simeq 0{,}33$ sur le test : le verrou de tout le projet n'est pas `cort` ou `dipg`, presque toujours bien classées, mais bien la classe rare `midl`.

Pour contrer ce déséquilibre dans la vraisemblance, nous utilisons des poids inverses de prévalence (Fig. 1B)

$$w_c = \frac{n}{K\,n_c}, \qquad w_{\text{cort}} = 0{,}87, \;\; w_{\text{dipg}} = 0{,}81, \;\; w_{\text{midl}} = 1{,}62,$$

qui multiplient la contribution de chaque patient `midl` par $\simeq 1{,}6$ (la dérivation de ces poids est détaillée en §3.3.1). Cette pondération est statistiquement plus défendable qu'un sur-échantillonnage synthétique de type SMOTE (que nous avons testé mais peu concluant) : elle rééquilibre la log-vraisemblance sans fabriquer de faux patients dans un espace à 16 931 dimensions, opération hautement instable en régime $p \ggg n$

Remarque 2.1 (Le « filet de sauvetage » de la classe rare). En décomposition One-vs-Rest, un classifieur binaire `midl`-vs-reste dont tous les coefficients seraient nuls conserve un intercept calibré sur la prévalence empirique. La probabilité plancher prédite vaut alors

$$\hat{P}_{\text{midl}} \simeq \frac{n_{\text{midl}}}{n} = \frac{8}{39} \simeq 0{,}205,$$

constante pour tous les patients. Si les modèles `cort` et `dipg` rejettent simultanément un patient, ce plancher peut suffire à le faire gagner par `argmax`. Ce mécanisme, déjà visible au stade de l'EDA, explique a posteriori pourquoi les pipelines OvR récupèrent 2/3 des `midl` là où la formulation multinomiale, contrainte par $\sum_c P_c = 1$, en récupère 0/3 (cf. section Résultats).

Une domination dimensionnelle du bloc GE. Le bloc GE concentre 92,7% des variables contre 7,3% pour CGH (Fig. 1C). Cette asymétrie a une conséquence directe sur les méthodes de fusion. Après standardisation variable par variable, la variance de chaque colonne vaut 1, donc l'inertie totale d'un bloc est simeqimativement égale à son nombre de variables p_j. L'inertie concaténée de $[\,\text{GE}\,|\,\text{CGH}\,]$ est donc dominée à plus de 90% par GE : toute méthode naïve sur la concaténation favorise mécaniquement GE et marginalise CGH. C'est précisément ce déséquilibre qui motive la remise à l'échelle $\tilde{X}_j = X_j/\sqrt{p_j}$ employée par la SGCCA (cf. section RGCCA et SGCCA), afin de ramener l'inertie de chaque bloc à 1.

2.2 Qualité des données et prétraitement

Valeurs manquantes et variance. L'inspection des deux blocs révèle des valeurs manquantes éparses et l'absence de variables de variance strictement nulle après imputation. Conformément au protocole du dépôt, nous imputons par la médiane calculée sur le train (puis appliquée au test), choix robuste aux valeurs extrêmes et qui n'utilise jamais l'information du test, évitant ainsi toute fuite.

Échelle des données GE. La distribution globale des $39 \times 15\,702$ valeurs d'expression (Fig. 2) est quasi symétrique, centrée (médiane = 0,03) et bornée (min = −10,43, max = 9,87) : il s'agit de niveaux d'expression déjà normalisés (log-ratios). Aucune transformation supplémentaire (log, Box-Cox) n'est donc nécessaire ; une simple standardisation z-score suffit en amont des méthodes sensibles à l'échelle (PCA, SGCCA).

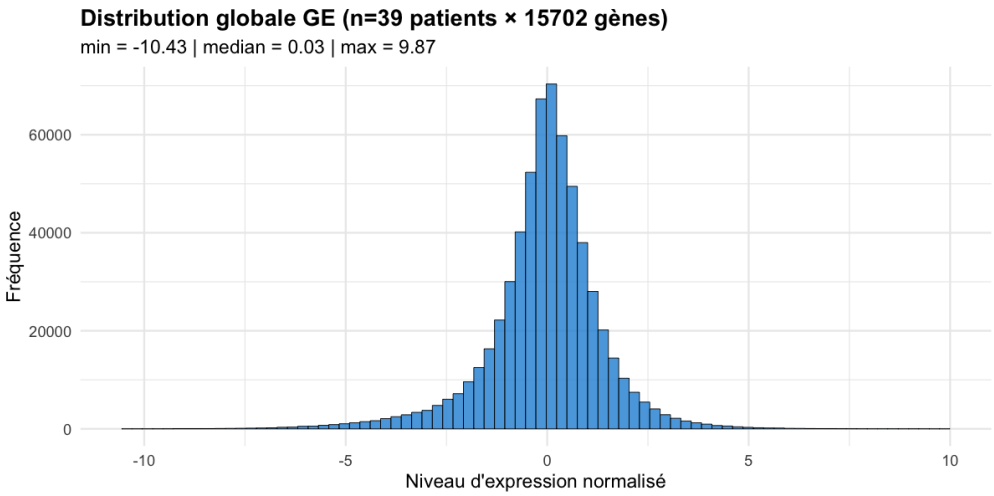


Fig. 2: Distribution globale des niveaux d'expression GE ($n = 39$ patients $\times$ 15 702 gènes). Le signal est centré et simeqimativement symétrique : données déjà normalisées, standardisation z-score suffisante.

Décalage train/test. Avec un test de seulement 14 patients, nous avons vérifié l'écart standardisé des moyennes (standardized mean difference, SMD) entre train et test, variable par variable, après standardisation par les statistiques du train. La distribution des |SMD| reste modérée pour la grande majorité des variables : il n'existe pas de covariate shift systématique massif, et la petite taille du test demeure la principale source d'incertitude — d'où le recours à une validation croisée répétée plutôt qu'à un simple split (§ Cross-Validation).

2.3 Séparabilité non supervisée : PCA par bloc

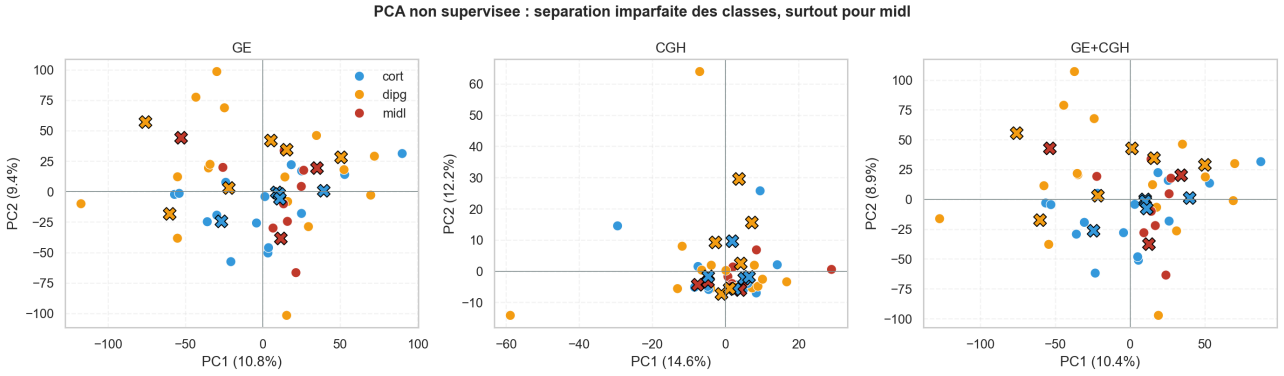


Fig. 3: PCA non supervisée (deux premières composantes) pour GE, CGH et la concaténation GE+CGH. Cercles : train ; croix : test. Les trois classes se recouvrent largement, en particulier `midl`, et les deux premières composantes ne capturent qu'une faible part de variance.

La PCA répond à une question simple : les classes sont-elles visibles sans supervision ? La réponse est négative (Fig. 3). Les deux premières composantes ne capturent qu'une faible part de l'inertie - PC1 + PC2 $\simeq$ 20% pour GE (10,8% + 9,4%), $\simeq$ 27% pour CGH (14,6% + 12,2%) - et surtout les nuages des trois classes se chevauchent fortement. La classe `midl` en particulier ne forme aucun amas distinct : ses points sont dispersés au milieu de `cort` et `dipg`.

Remarque 2.2. Cette absence de structure non supervisée a une conséquence méthodologique forte

: une réduction de dimension non supervisée en amont (PCA, Sparse PCA) jette l'essentiel de la variance dans des directions non discriminantes pour `midl`. Cela explique a priori l'échec des pipelines `Sparse PCA + LogReg` que nous avons testés (résultats désastreux, non rapportés). À l'inverse, une projection supervisée (SGCCA, qui corrèle chaque bloc au bloc des labels) est indispensable pour faire émerger les directions discriminantes.

2.4 Structure de corrélation et redondance

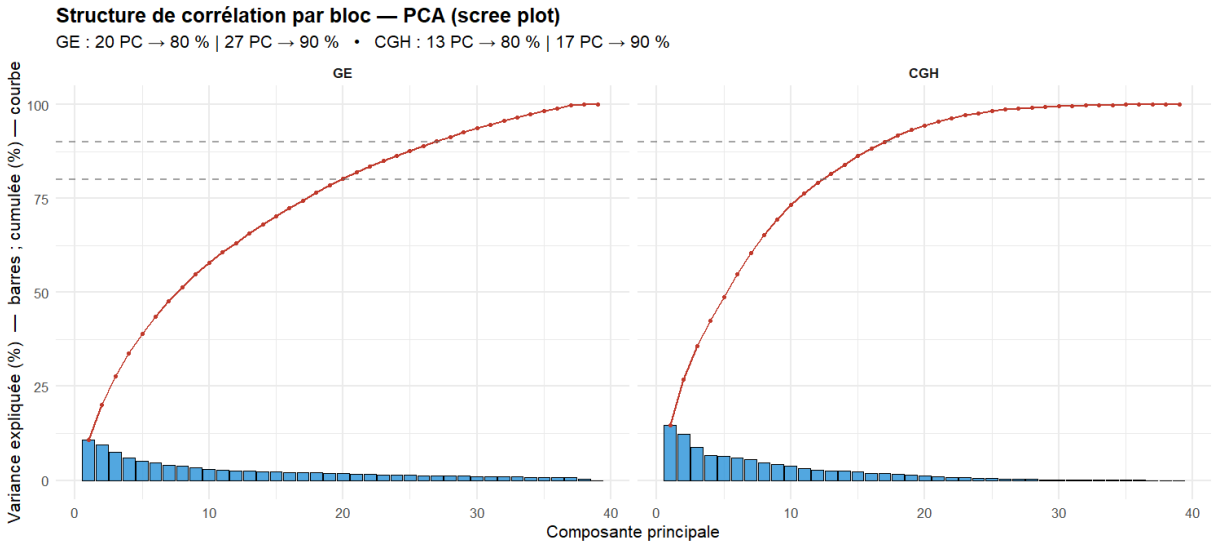


Fig. 4: Scree plot par bloc : variance expliquée par composante (barres) et variance cumulée (courbe rouge), seuils 80% et 90% en pointillés. En régime $p \ggg n$, le rang est borné par $n - 1 = 38$: la vitesse de concentration de la variance mesure la redondance (collinéarité) intra-bloc et la dimensionnalité effective.

En régime $p \ggg n$, une part importante des variables est redondante. Plutôt qu'une matrice de corrélation brute sur un échantillon de variables tirées au hasard (illisible sur 15 702 / 1 229 colonnes et sans ordre interprétable) nous résumons cette structure par la PCA de chaque bloc (Fig. 4) si la variance se concentre dans peu de composantes, c'est que les variables sont fortement corrélées. Le constat est net. Les deux blocs ont une dimensionnalité effective très inférieure à leur nombre de variables (rang ≤ 38) : il faut 20 composantes pour capturer 80% de la variance GE (27 pour 90%), contre seulement 13 pour CGH (17 pour 90%). La redondance est donc plus marquée dans CGH, ce qui est cohérent avec des segments chromosomiques contigus, donc co-altérés tandis que GE répartit sa variance sur davantage de directions. Cette redondance a deux implications directes :

- elle justifie une pénalité Elastic-Net plutôt que Lasso pur : la composante ℓ^2 stabilise la sélection parmi des variables quasi colinéaires, là où le Lasso en choisirait une au hasard ;
- elle éclaire la divergence numérique de l'algorithme de Newton observée pour le cooperative learning à $\rho > 0$: la matrice $X^\top W X$ est déjà quasi singulière, et le terme d'accord $\rho X^\top X$ amplifie d'un facteur $(1 + \rho)$ des corrélations intra-bloc déjà extrêmes (cf. section Résultats).

2.5 Signal univarié autour de `midl`

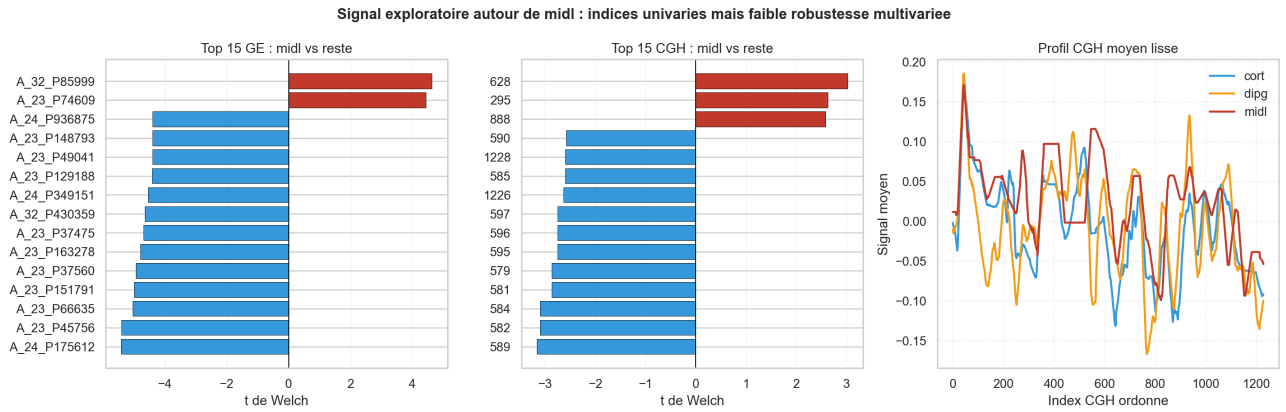


Fig. 5: À gauche et au centre : top 15 des variables GE et CGH par $|t|$ de Welch pour le contraste `midl` vs reste (rouge : sur-exprimé chez `midl`). À droite : profil CGH moyen lissé par classe le long de l'index génomique ordonné.

Pour évaluer s'il existe un signal exploitable pour la classe rare, nous calculons des t de Welch univariés `midl`-vs-reste (Fig. 5). Le test de Welch est préféré au t de Student classique car il ne suppose pas l'égalité des variances entre groupes, hypothèse intenable avec $n_{\text{midl}} = 8$ contre 31 pour le reste.

- **GE** : quelques gènes atteignent $|t| \simeq 4$–5 (p. ex. `A_32_P85999`, sur-exprimé chez `midl`), tandis qu'une majorité de gènes y sont sous-exprimés. Un signal univarié existe donc bien.
- **CGH** : les statistiques plafonnent vers $|t| \simeq 3$, signal plus faible, cohérent avec le fait que les modèles parcimonieux pilotés par les données ne sélectionnent presque jamais CGH.
- **Profil CGH** : les courbes moyennes par classe se chevauchent largement le long de l'index génomique (Fig. 5, droite) ; aucune région n'isole nettement la classe rare.

Remarque 2.3 (Univarié $\neq$ multivarié robuste). La présence d'un signal univarié ne garantit pas une discrimination multivariée robuste. Avec $p \ggg n$ et $n_{\text{midl}} = 8$, ces t-statistiques sont elles-mêmes très bruitées (un seul patient peut faire basculer le classement) et ne contrôlent pas la multiplicité ($\sim 17\,000$ tests). Ces diagnostics indiquent où chercher, mais ne constituent pas une sélection valide : c'est pourquoi la sélection finale est confiée à des méthodes pénalisées intra-CV.

2.6 Choix de la métrique d'évaluation

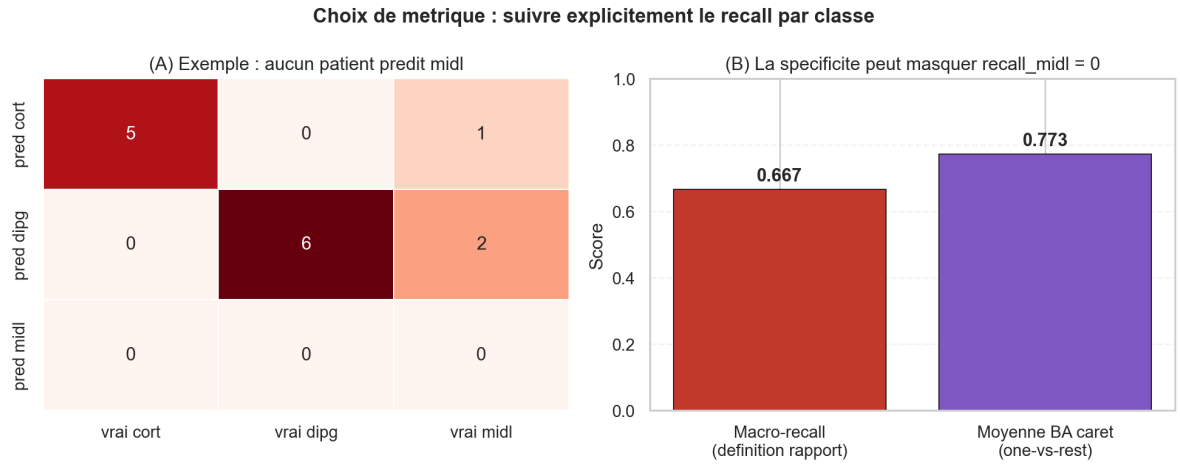


Fig. 6: **(A)** Matrice de confusion d'un classifieur ne prédisant jamais `midl` ($\text{recall}_{\text{midl}} = 0$). **(B)** La même prédiction obtient un macro-recall de 0,667 mais une « balanced accuracy » moyenne one-vs-rest (définition `caret`) de 0,773, qui masque l'échec total sur `midl`.

Le déséquilibre de classes impose un choix de métrique explicite. L'accuracy brute pondère implicitement les classes par leur fréquence et masquerait un échec sur `midl` : un classifieur ne prédisant jamais `midl` atteindrait déjà $\simeq 11/14$ sur le test. Nous retenons donc le **macro-recall** (moyenne des recalls par classe, cf. la balanced accuracy de la section Cross-Validation), qui attribue le même poids aux trois classes.

Un piège subtil mérite d'être signalé (Fig. 6). Dans plusieurs scripts `R`, la moyenne des colonnes `"Balanced Accuracy"` renvoyées par `caret` (`mean(cm$byClass[,...])`) calcule la moyenne des balanced accuracies one-vs-rest, qui incorporent la spécificité. Or la spécificité d'une classe rare reste élevée même quand son recall est nul. Sur l'exemple de la Fig. 6, où aucun patient n'est prédit `midl` :

$$\text{macro-recall} = \tfrac{1}{3}\left(\tfrac{5}{5} + \tfrac{6}{6} + \tfrac{0}{3}\right) = 0{,}667, \qquad \text{BA caret (OvR)} = 0{,}773.$$

La métrique `caret` surévalue donc le modèle de +0,1 en masquant $\text{recall}_{\text{midl}} = 0$. Nous suivons par conséquent explicitement le recall par classe et la matrice de confusion, et non la seule BA agrégée.

2.7 Implications pour la modélisation

Cette analyse exploratoire des données nous renseigne sur les problèmes fondamentaux de ce projet :

- Le régime $p \ggg n$: impose une régularisation importante et de la sélection de variable en plus de la pénalisation ℓ^1 .
- Le verrou de la classe minoritaire : le déséquilibre impose des stratégies de "rééquilibrage" (métriques adaptées, pondération de la vraisemblance, etc.)
- La projection non supervisée ne sépare pas les classes $\Longrightarrow$ attire l'oeil vers de la projection supervisée (LDA, SGCCA).
- Domination structurel du bloc GE (92.7% des variables et de l'inertie standardisée) $\Longrightarrow$ Nous avons du travailler avec les deux blocs (GE, CGH) de manière appropriée (SGCCA).

3 Formalisme mathématique

3.1 Notion d'entropie croisée[6]

L'entropie croisée est une mesure mathématique permettant de quantifier la différence entre deux distributions de probabilités. De manière générale, si l'on considère une distribution de probabilité « vraie » (ou cible) P et une distribution « estimée » (ou simeqimée) Q sur un même espace d'événements discret $\mathcal{X}$, l'entropie croisée de Q par rapport à P se définit par :

$$H(P, Q) = -\sum_{x \in \mathcal{X}} P(x) \log Q(x)$$

Conceptuellement, elle représente le nombre moyen de bits (si l'on utilise un logarithme en base 2) nécessaires pour coder un événement tiré de la vraie distribution P en utilisant un code optimisé pour la distribution estimée Q. Minimiser l'entropie croisée revient exactement à minimiser la divergence de Kullback-Leibler

$$D_{KL}(P||Q) = -\sum_{x \in \mathcal{X}} P(x) \log \frac{Q(x)}{P(x)}$$

3.2 Modélisation du problème[7]

Nous sommes dans un problème de classification à 3 classes. On observe un dataset d'entraînement $\mathcal{D} := \{(X_1, Y_1), \dots, (X_n, Y_n)\}$ avec $n = 39$, où les points X_i sont les données et Y_i est le label associé à X_i.
Les X_i appartiennent à l'espace de feature $\mathcal{X} = \mathbb{R}^p$ avec $p = \dots$, tandis que les labels Y_i sont dans $\mathcal{Y} = \{1, 2, 3\}$. Dans un premier temps nous omettons le fait que les features sont distribuées selon deux blocs distincts.
On suppose les (X_i, Y_i) i.i.d. distribués selon une loi inconnue $\mathbb{P}$.
On appelle classifieur une fonction $f : \mathcal{X} \to \mathcal{Y}$. Une fonction de loss est une fonction $\ell : \mathcal{Y}^2 \to \mathbb{R}$. On appelle le risque d'un classifieur f selon la loss ℓ la quantité

$$\mathcal{R}(f) := \mathbb{E}_{(X,Y)\sim\mathbb{P}}\big[\, \ell(f(X), Y)\big]$$

Cette quantité est inconnue. En pratique, on divise le dataset $\mathcal{D}$ en :

- Un ensemble d'entraînement ($\simeq 80\%$ de l'ensemble) utilisé pour déterminer le prédicteur f
- Un ensemble de test $\mathcal{D}'_m = \{(X'_1, Y'_1), \dots, (X'_m, Y'_m)\}$ est utilisé pour estimer le risque via la loi forte des grands nombres

$$\frac{1}{m} \sum_{i=1}^{m} \ell(f(X'_i), Y'_i) \underset{m \to +\infty}{\longrightarrow} \mathcal{R}(f) \text{ p.s.}$$

Enfin, on appelle prédicteur bayésien

$$f^* = \operatorname*{argmin}_f \mathcal{R}(f)$$

Et on note $\mathcal{R}^* = \mathcal{R}(f^*)$.

[6] Cette section s'appuie sur le cours de théorie de l'information, CentraleSupélec.
[7] Cette section s'appuie sur le cours de machine learning théorique, ENSAE.

Pour une classe $c \in \mathcal{Y}$, on définit la probabilité conditionnelle pour chaque classe c :

$$\eta_c(x) = \mathbb{P}[Y = c|X = x]$$

Sous la fonction de loss 0-1, $\ell(y, y') = \mathbf{1}_{y \neq y'}$, le risque devient alors le taux d'erreur de classification, et on a :

$$f^*(x) = \underset{c \in \mathcal{Y}}{\text{argmax}}\ \eta_c(x)$$

En effet : soit un classifieur f. Sous la 0-1 loss, le risque s'écrit

$$\begin{aligned}\mathcal{R}(f) = \mathbb{E}_{(X,Y)}[\ell(f(X), Y)] &= \mathbb{E}_{(X,Y)}[\mathbf{1}_{f(X) \neq Y}] \\ &= \mathbb{E}_X\left[\mathbb{E}_{Y|X}[\mathbf{1}_{f(X) \neq Y}]\right] \\ &= \mathbb{E}_X[\mathbb{P}[f(X) \neq Y|X]]\end{aligned}$$

Puisque $\mathbb{P} \geqslant 0$, il suffit de minimiser le terme à l'intérieur de l'espérance pour chaque observation $x \in \mathcal{X}$ fixée. Pour un x donné, on note $\hat{y} = f(x)$ la classe prédite. La probabilité d'erreur est

$$\mathbb{P}[\hat{y} \neq Y|X = x] = 1 - \mathbb{P}[\hat{y} = Y|X = x]$$

On doit alors maximiser $\mathbb{P}[\hat{y} = Y|X = x]$, d'où le résultat.

Remarque : ce résultat est spécifique à la 0-1 loss. Sous une loss asymétrique (par exemple pour pénaliser davantage la mauvaise classification de `midl`), le prédicteur bayésien ne s'écrirait plus comme l'argmax des η_c.

3.3 Régression logistique multiclasse[8]

Sous la fonction de loss 0-1, le prédicteur bayésien s'écrit :

$$f^*(x) = \text{argmax}_{c \in \mathcal{Y}} \eta_c(x), \quad \text{où } \eta_c(x) = \mathbb{P}[Y = c|X = x]$$

Cependant, la 0-1 loss n'étant ni continue ni dérivable, la minimisation directe de son risque empirique est un problème NP-difficile. Pour contourner cette difficulté, la régression logistique multinomiale utilise une fonction de perte de substitution convexe et dérivable : la cross-entropy.

Dans le modèle logistique, on suppose que η_c peut s'écrire, pour toute classe c :

$$\eta_c^\beta(x) = \mathbb{P}_\beta[Y = c|X = x] = \frac{e^{\beta_c^T x}}{\sum_{k=1}^3 e^{\beta_k^T x}}$$

avec $\beta = (\beta_1, \beta_2, \beta_3) \in \mathbb{R}^{p \times 3}$ et une matrice de paramètres.

Le modèle statistique ci-dessus est sur-paramétré (donc non identifiable) : en remplaçant β_c par $\beta_c + \gamma$ pour tout $c \in \mathcal{Y}$ et $\gamma \in \mathbb{R}^p$, alors les probabilités $\eta_c^\beta(x)$ ne changent pas. Pour lever cette « ambiguité », on fixe conventionnellement $\beta_3 = 0$.

Les observations étant i.i.d., la vraisemblance du modèle s'écrit :

$$\mathcal{L}(\beta) = \prod_{i=1}^n \prod_{c=1}^3 (\eta_c^\beta(X_i))^{\mathbf{1}_{Y_i = c}}$$

[8] Cette section s'appuie sur les cours de machine learning théorique et d'optimisation, ENSAE.

En passant au logarithme, la log-vraisemblance $\ell(\beta)$ devient :

$$\ell(\beta) = \sum_{i=1}^{n} \underbrace{\left(\beta_{Y_i}^T X_i - \log \left(\sum_{k=1}^{3} e^{\beta_k^T X_i} \right) \right)}_{:=\ell_i(\beta)}$$

Maximiser cette log-vraisemblance revient très exactement à minimiser la cross-entropy empirique. Le risque empirique que nous cherchons à minimiser s'écrit donc :

$$\hat{R}_n(\beta) = -\frac{1}{n} \sum_{i=1}^{n} \log \eta_{Y_i}^{\beta}(X_i)$$

La fonction $\beta \mapsto \hat{R}_n(\beta)$ est convexe et différentiable : très pratique pour l'optimisation.

3.3.1 Vraisemblance pondérée et choix des poids de classe

Le risque empirique $\hat{R}_n$ ci-dessus accorde le même poids $1/n$ à chaque patient. Or, comme l'analyse exploratoire l'a montré (§2), les classes sont déséquilibrées : la classe `midl` ne représente que $n_{\text{midl}}/n = 8/39 \simeq 20{,}5\%$ de l'entraînement. Minimiser $\hat{R}_n$ revient alors à minimiser une erreur dominée par les classes majoritaires, et la solution triviale qui ignore `midl` obtient déjà une faible perte. Pour aligner l'objectif d'apprentissage sur la métrique d'évaluation (la balanced accuracy, qui pondère les classes également), nous introduisons une **log-vraisemblance pondérée** :

$$\hat{R}_n^w(\beta) \;=\; -\frac{1}{n} \sum_{i=1}^{n} w_{Y_i} \, \log \eta_{Y_i}^{\beta}(X_i),$$

où chaque observation reçoit un poids $w_{Y_i} > 0$ qui ne dépend que de sa classe.

Dérivation du poids inverse de prévalence. On souhaite que la contribution cumulée de chaque classe au risque soit identique, quelle que soit sa taille. La classe c regroupe n_c patients, dont chacun pèse w_c ; sa contribution totale est donc proportionnelle à $w_c \, n_c$. Imposer l'égalité des contributions entre les K classes,

$$w_c \, n_c \;=\; \text{constante} \quad \forall c,$$

puis normaliser de sorte que le poids moyen vaille 1 (i.e. $\sum_c w_c \, n_c = n$, ce qui préserve l'échelle globale du risque), conduit à

$$\boxed{w_c \;=\; \frac{n}{K \, n_c}}.$$

On retrouve exactement les valeurs annoncées dans l'EDA :

$$w_{\text{cort}} = \frac{39}{3 \times 15} = 0{,}87, \qquad w_{\text{dipg}} = \frac{39}{3 \times 16} = 0{,}81, \qquad w_{\text{midl}} = \frac{39}{3 \times 8} = 1{,}62.$$

Chaque patient `midl` pèse ainsi $\simeq 1{,}6$ fois un patient « moyen », et environ deux fois un patient `dipg`. Ce schéma est celui appelé `class_weight="balanced"` dans la plupart des bibliothèques, et il revient à pondérer la log-vraisemblance par l'inverse de la fréquence empirique de la classe.

Interprétation. Trois lectures équivalentes éclairent ce choix. (i) Ré-échantillonnage implicite : pondérer la classe c par $w_c \propto 1/n_c$ équivaut, en espérance, à ré-échantillonner les données jusqu'à équilibrer les effectifs, mais sans dupliquer ni créer d'observation. (ii) Correction de la prévalence : si la prévalence d'entraînement diffère de la prévalence cible (ici uniforme, puisque la balanced accuracy

traite les classes à parité), la pondération corrige ce décalage dans la vraisemblance. (iii) Cohérence objectif/métrique : le gradient pondéré

$$\nabla_{\beta_c} \hat{R}_n^w(\beta) = \frac{1}{n} \sum_{i=1}^{n} w_{Y_i} \left(\eta_c^\beta(X_i) - \mathbf{1}_{\{Y_i=c\}} \right) X_i$$

amplifie d'un facteur w_{midl} les résidus des patients `midl`, forçant le modèle à ne plus les négliger. La pondération laisse le problème convexe et différentiable : elle ne change donc rien aux propriétés d'optimisation, seulement le point optimal visé.

Combinaison avec la régularisation. Les poids agissent sur le terme d'attache aux données ; ils se composent sans difficulté avec la pénalité Elastic-Net introduite plus loin, l'objectif complet devenant $\hat{R}_n^w(\beta) + \lambda \,\text{pen}(\beta)$. En pratique, dans les pipelines reposant sur `glmnet`, ces poids sont passés via l'argument `weights` (poids par observation w_{Y_i}), tandis que la pénalité reste pilotée par λ et α.

Pour déterminer le gradient de $\hat{R}_n$, nous devons calculer les dérivées partielles par rapport à chaque vecteur de paramètres β_c (pour $c \in \{1, 2, 3\}$). En utilisant la règle de la chaîne sur la fonction softmax, on obtient :

$$\nabla_{\beta_c} \ell_i(\beta) = \left(\eta_c^\beta(X_i) - \mathbf{1}_{\{Y_i=c\}} \right) X_i$$

En sommant sur l'ensemble des observations, le gradient complet de notre fonction objectif s'exprime ainsi :

$$\nabla_{\beta_c} \hat{R}_n(\beta) = \frac{1}{n} \sum_{i=1}^{n} \left(\eta_c^\beta(X_i) - \mathbf{1}_{\{Y_i=c\}} \right) X_i$$

On remarque que le gradient est proportionnel à l'erreur de prédiction $(\eta_c^\beta(X_i) - \mathbf{1}_{\{Y_i=c\}})$, pondérée par la valeur des variables explicatives X_i. Le modèle est calibré lorsque la probabilité moyenne prédite pour chaque classe coïncide exactement avec sa fréquence observée conditionnellement aux features.

Cependant, l'équation $\nabla_{\beta_c} \hat{R}_n(\beta) = 0$ est non linéaire et ne possède pas de solution analytique fermée. Nous devons alors utiliser des algorithmes d'optimisation : descente de gradient, méthode de Newton, etc.

Au lieu d'utiliser une Descente de gradient standard (qui nécessite de calculer la somme sur les n observations à chaque itération), nous privilégions en pratique une Descente de gradient stochastique SGD. À chaque étape t, la SGD estime le gradient à partir d'une seule observation tirée au hasard (ou d'un mini-batch), ce qui réduit considérablement le coût de calcul.

L'équation de mise à jour des paramètres à l'itération $t+1$ pour une observation i choisie aléatoirement s'écrit :

$$\beta_c^{(t+1)} = \beta_c^{(t)} - \gamma_t \nabla_{\beta_c} \ell_i(\beta^{(t)})$$

$$\beta_c^{(t+1)} = \beta_c^{(t)} - \gamma_t \left(\eta_c^{\beta^{(t)}}(X_i) - \mathbf{1}_{\{Y_i=c\}} \right) X_i$$

Dans cette formulation, $\gamma_t > 0$ est hyperparamètre. Il contrôle la taille du pas effectué dans la direction opposée au gradient. S'il est trop grand, l'algorithme risque de diverger ou d'osciller autour du minimum ; s'il est trop petit, la convergence sera extrêmement lente.

Une fois l'algorithme convergé (minimum atteint ou alors critère d'arrêt de l'algorithme), nous obtenons la matrice des estimateurs optimaux $\hat{\beta}$. Pour une nouvelle observation x (appartenant à notre ensemble de test $\mathcal{D}'_m$), l'objectif est d'inférer sa classe. Conformément au paragraphe précédent, cette prédiction est donnée par :

$$\hat{y} = \text{argmax}_{c \in \mathcal{Y}} \eta_c^{\hat{\beta}}(x)$$

En remplaçant les probabilités $\eta_c^{\hat{\beta}}(x)$:

$$\hat{y} = \text{argmax}_{c\in\mathcal{Y}} \frac{e^{\hat{\beta}_c^T x}}{\sum_{k=1}^{3} e^{\hat{\beta}_k^T x}}$$

Le dénominateur de la fonction softmax est strictement positif et reste constant quelle que soit la classe c évaluée pour un x donné. Par ailleurs, la fonction exponentielle étant strictement croissante, elle conserve l'ordre des valeurs. Par conséquent, déterminer l'argument qui maximise la probabilité revient simplement à déterminer l'argument qui maximise le terme dans l'exponentielle. La prédiction de la classe se réduit alors à la maximisation du score linéaire (ou logit) :

$$\hat{y} = \text{argmax}_{c\in\{1,2,3\}} (\hat{\beta}_c^T x)$$

Finalement, la frontière de décision finale qui sépare les classes dans l'espace $\mathbb{R}^p$ reste purement linéaire.

3.4 Minimisation du risque empirique pénalisé : Elastic-Net[9]

Pour contrôler la complexité du modèle, et éviter tout régime d'overfitting, nous introduisons une pénalisation dans la fonction objectif :

$$\min_{f\in\mathcal{F}} \hat{\mathcal{R}}_n(f) + \lambda \times \text{pen}(f)$$

Le paramètre $\lambda > 0$ gère le compromis entre l'ajustement aux données d'entraînement et la simplicité du modèle.

Dans notre projet, nous appliquons une pénalisation de type Elastic-Net, qui combine de manière convexe deux termes de régularisation :

- Une régularisation Ridge donnée par la norme ℓ^2 : le terme $||\beta||_2^2$ limite l'amplitude des coefficients, ce qui est particulièrement efficace en présence de variables explicatives fortement corrélées.
- Une régularisation Lasso donnée par la norme ℓ^1 : le terme $||\beta||_1$ induit de la parcimonie (sparsity) en forçant les coefficients des variables les moins pertinentes à être exactement nuls, opérant ainsi une sélection de variables intégrée au modèle.

En paramétrant cet équilibre par $\rho \in [0, 1]$, le problème d'optimisation final de notre modèle se formule ainsi

$$\min_{\beta} \left(\hat{R}_n(\beta) + \lambda\rho||\beta||_1 + \frac{\lambda(1-\rho)||\beta||_2^2}{2} \right)$$

Notons un problème : la fonction $\beta \mapsto ||\beta||_1$ n'est pas différentiable partout. La mise à jour de β présentée précédemment est donc remplacée par des algorithmes d'optimisation proximale, qui appliquent à chaque itération un opérateur de seuillage doux pour forcer l'annulation exacte de certains coefficients.

3.5 GLM[10]

Un modèle linéaire généralisé (GLM) modélise la loi conditionnelle $Y_i|X_i = x_i$ via trois objets distincts. On suppose que les $Y_i|X_i = x_i$ suivent une loi de la famille exponentielle, de densité :

$$f_{Y_i}(y_i|\theta_i, \phi_i) = \exp\left(\frac{y_i\theta_i - b(\theta_i)}{a(\phi_i)} + c(y_i, \phi_i) \right)$$

[9] Cette section s'appuie sur le cours de statistiques, CentraleSupélec.

[10] Cette section s'appuie sur les cours de Big Data & Santé (CentraleSupélec) et de machine learning théorique (ENSAE).

où :

- θ_i est le paramètre canonique.
- ϕ_i est un paramètre de dispersion.
- a, b, c sont des fonctions caractéristiques de la famille.

On a :

$$\mathbb{E}[Y_i|X_i = x_i] = b'(\theta) := \mu$$

Si b est monotone (c'est le cas en pratique) : $\theta = g(\mu)$ avec g appelée la fonction de lien canonique.

$$\mathrm{Var}(Y_i|X_i = x) = b''(\theta)a(\phi) = V(\mu)a(\phi)$$

La variance n'est donc pas un paramètre libre : elle est dictée par la moyenne via la fonction V.

Hypothèse : on suppose que l'effet de x sur la loi de Y passe uniquement par une combinaison linéaire de ses composantes :

$$g(\mu_i) = x_i^T\beta := \eta$$

où $\beta \in \mathbb{R}^p$ est un paramètre à estimer.
La fonction de lien g est une bijection monotone qui relie la moyenne μ au prédicteur linéaire η :

$$\mu_i = g^{-1}(x_i^T\beta)$$

Par défaut, on choisit g de sorte que $\eta = \theta$, pour la concavité. On parle de lien canonique. On notera :

- En régression linéaire avec lien identité, $\mu = \eta = X^T\beta$ et donc la prédiction est égale au prédicteur linéaire
- Cependant, dès qu'on quitte le modèle gaussien, les objets se séparent :

$$\eta = X^T\beta \xrightarrow{g^{-1}} \mu = g^{-1}(\eta) = \text{ prédiction du modèle}$$

- La régression logistique est l'exemple canonique de GLM avec lien non-trivial.

Développons le dernier point. Plaçons nous dans le cas $K = 2$ classes pour le moment.
On suppose que $Y|X = x \sim \mathcal{B}(\mu(x))$ avec $\mu(x) = \mathbb{P}[Y = 1|\ X = x] = \mathbb{E}[Y|X = x] \in (0, 1)$. La densité s'écrit :

$$f_Y(y|x) = \mu(x)^y(1-\mu(x))^{1-y} = \exp\left(y\underbrace{\log\frac{\mu(x)}{1-\mu(x)}}_{:=\theta(x)} + \underbrace{\log(1-\mu)}_{=-b(\theta(x))}\right)$$

C'est une famille exponentielle avec :

- Paramètre canonique $\theta(x) = \log\dfrac{\mu(x)}{1+\mu(x)}$ (on notera également θ_x).
- $b(\theta_x) = \log(1 + e^{\theta_x})$
- $a(\phi) = 1$ (pas de dispersion)

On vérifie bien que $\mathbb{E}[Y|X=x] = b'(\theta_x) = \dfrac{e^{\theta_x}}{1+e^{\theta_x}} = \mu$ et $\mathrm{Var}(Y|X=x) = b''(\theta_x) = \mu(x)(1-\mu(x))$.

On postule la linéarité au niveau du paramètre canonique :

$$\theta(x) = \eta(x) = x^T\beta$$

On a alors :

$$\log\frac{\mu(x)}{1-\mu(x)} = \eta(x) = x^T\beta$$

On obtient un lien logit : $\mathrm{logit}(\mu(x)) = x^T\beta$. En inversant :

$$\mu(x) = \frac{1}{1+e^{-\eta}} = \sigma(x^T\beta)$$

avec σ la sigmoide. On retrouve exactement la formule de la régression logistique binaire.La log-vraisemblance étant strictement concave en η (et donc en β), l'estimateur du maximum de vraisemblance est unique et calculable par méthode de Newton.
Dans le cas $K \geqslant 2$ classes, il faut pousser encore un peu. On suppose que $Y|X=x$ suit une loi catégorielle $\mathrm{Cat}(\mu_1(x),\dots,\mu_K(x))$ avec $\sum_{c=1}^{K}\mu_c = 1$, de densité :

$$f_Y(y|x) = \prod_{c=1}^{K}\mu_c(x)^{\mathbf{1}_{y=c}}$$

En vectorisant Y en one-hot $(Y_1,\dots,Y_K)$ (définition en-dessous) :

$$f_Y(y|x) = \prod_{c=1}^{K}\mu_c(x)^{y_c} = \exp\Big(\sum_{c=1}^{K-1} y_c \underbrace{\log\frac{\mu_c(x)}{\mu_K(x)}}_{:=\theta_c(x)} + \log(\mu_K(x))\Big)$$

Note : quand $Y \in \{1,\dots,K\}$ est une variable catégorielle on remplace la valeur scalaire par un vecteur de $\mathbb{R}^K$ qui contient un seul 1 (la position de la classe vraie) et $K-1$ zéros. On transforme donc Y en $(0,\dots,1,\dots,0)$.

On obtient alors une famille exponentielle mais vectorielle : $\theta = (\theta_1,\dots,\theta_K)$. La classe K sert de référence, c'est la même contrainte d'identifiabilité $\beta_K = 0$ expliquée dans la section précédente.
On suppose la linéarité au niveau de chaque coordonnée du paramètre canonique :

$$\theta_c(x) = \eta_c(x) = x^T\beta_c$$

D'où la matrice de paramètres $\beta \in \mathbb{R}^{p\times K}$ avec $\beta_K = 0$. Finalement, on obtient :

$$\mu_c(x) = \frac{e^{x^T\beta_c}}{1+\sum_{c'=1}^{K-1}e^{x^T\beta_{c'}}} = \mathrm{softmax}_c(x^T\beta)$$

où $\mathrm{softmax}_c : z \mapsto \dfrac{e^{z_c}}{\sum_{k=1}^{K}e^{z_k}}$
On retombe alors sur le cadre de la section précédente.

3.6 Linear Discriminant Analysis (LDA)

On rappelle que le classifieur bayésien est celui qui maximise la probabilité a posteriori

$$f^*(x) = \underset{c\in\mathcal{Y}}{\operatorname{argmax}}\ \mathbb{P}[Y = c|X = x]$$

On peut réécrire cette probabilité :

$$\mathbb{P}[Y = c|X = x] = \frac{\mathbb{P}[X = x|Y = c]\mathbb{P}[Y = c]}{\mathbb{P}[X = k]}$$

Et alors :

$$f^*(x) = \underset{c\in\mathcal{Y}}{\operatorname{argmax}}(\pi_c f_c(x))$$

où $\pi_c = \mathbb{P}[Y = c]$ est la probabilité a priori de la classe c, et $X|Y = c \sim P_c$ tel que $dP_c = f_c d\lambda^{(p)}$.
Les hypothèses du formalisme LDA sont les suivantes :

- Normalité : Conditionnellement à la classe c, les données suivent une loi normale multivariée en dimension p

$$X|Y = c \sim \mathcal{N}_p(\mu_p, \Sigma_p)$$

- Homoscédasticité :

$$\forall c \in \mathcal{Y},\ \Sigma_c = \Sigma$$

Posons $\delta_c(x) = \log(\pi_c f_c(x)) = \log \pi_c + \log f_c(x)$. En injectant la densité gaussienne :

$$\delta_c(x) = \log(\pi_c) - \frac{d}{2}\log(2\pi) - \frac{1}{2}\log(\det\Sigma) - \frac{1}{2}(x-\mu_c)^T\Sigma^{-1}(x-\mu_c)$$

Le terme quadratique se développe, par symétrie de Σ :

$$(x-\mu_c)^T\Sigma^{-1}(x-\mu_c) = x^T\Sigma^{-1}x + \mu_c^T\Sigma^{-1}\mu_c - 2x^T\Sigma^{-1}\mu_c$$

Et alors

$$\delta_c(x) = -\frac{1}{2}x^T\Sigma^{-1}x + x^T\Sigma^{-1}\mu_c - \frac{1}{2}\mu_c^T\Sigma^{-1}\mu_c + \log(\pi_c) + \text{constantes indépendantes de } c$$

On peut ignorer les termes ne dépendant pas de c :

$$\delta_c(x) = \underbrace{(\Sigma^{-1}\mu_c)^T}_{\mathbf{w}_c^T} x \underbrace{-\frac{1}{2}\mu_c^T\Sigma^{-1}\mu_c + \log(\pi_c)}_{b_c}$$

La fonction discriminante équivalente devient strictement linéaire en x ! La règle de décision finale est donc un hyperplan :

$$f^*(x) = \underset{c\in\mathcal{Y}}{\operatorname{argmax}}(\mathbf{w}_c^T x - b_c)$$

À présent, soit $n_c = \sum_{i=1}^{n} \mathbf{1}_{Y_i=c}$ le nombre d'observations de la classe c. On estime les paramètres inconnus (π_c, μ_c, Σ) :

$$\hat{\pi}_c = \frac{n_c}{n}$$

$$\hat{\mu}_c = \frac{1}{n_c} \sum_{i=1}^{n} X_i \mathbf{1}_{Y_i=c}$$

$$\hat{\Sigma} = \frac{1}{n-3} \sum_{c=1}^{3} \sum_{i=1}^{n} \left(X_i - \hat{\mu}_c\right)\left(X_i - \hat{\mu}_c\right)^T \mathbf{1}_{Y_i=c}$$

Note sur la covariance : Grâce à l'hypothèse d'homoscédasticité, on cherche une unique matrice commune à toutes les classes à partir de tout le jeu de données. Si on isole la classe c, la dispersion des données autour de leur moyenne de classe $\hat{\mu}_c$ est capturée par la matrice des sommes des carrés $S_c = \sum_{i=1}^{n} (X_i - \hat{\mu}_c)(X_i - \hat{\mu}_c)^T \mathbf{1}_{Y_i=c}$. Puisque la covariance est supposée identique dans chaque classe, on additionne simplement toutes ces matrices de dispersions pour obtenir le résultat. Pour obtenir l'estimateur final, on normalise en divisant par $n - K$ ($K = 3$) pour obtenir un estimateur non-biaisé (sinon, on peut aussi prendre l'estimateur du maximum de vraisemblance en divisant simplement par n mais l'estimateur est biaisé)

On remplace ensuite les vrais paramètres par leurs estimateurs dans l'équation de $\delta_k(x)$ pour obtenir notre classifieur entraîné $f(x)$.

3.7 Cross-Validation

La taille modeste de notre ensemble d'entraînement ($n = 39$) interdit tout estimateur naïf du risque par split unique. Nous recourrons donc à une validation croisée stratifiée répétée, complétée par une CV imbriquée pour la sélection d'hyperparamètres.

K-fold cross-validation. Soit $\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^n$ l'ensemble d'entraînement et $\mathcal{A}$ un algorithme d'apprentissage paramétré par un hyperparamètre θ. On partitionne $\{1, \dots, n\}$ en K sous-ensembles disjoints $\mathcal{F}_1, \dots, \mathcal{F}_K$ de tailles simeqimativement égales. Pour chaque pli k, on entraîne $\hat{f}_\theta^{(-k)} := \mathcal{A}(\mathcal{D} \setminus \mathcal{F}_k, \theta)$ sur les $K-1$ autres folds, puis on évalue sur $\mathcal{F}_k$. L'estimateur CV du risque s'écrit :

$$\widehat{\mathcal{R}}_{\mathrm{CV}}(\theta) \;=\; \frac{1}{K}\sum_{k=1}^{K} \frac{1}{|\mathcal{F}_k|} \sum_{i \in \mathcal{F}_k} \ell\Big(\hat{f}_\theta^{(-k)}(X_i),\, Y_i\Big)\,.$$

Stratification. La rareté de la classe `midl` ($n_{\mathrm{midl}} = 8$, soit à peine 20% du train) impose une stratification : on construit les plis de telle sorte que chaque $\mathcal{F}_k$ préserve la distribution empirique des classes,

$$\forall k \in \{1, \dots, K\},\;\; \forall c \in \mathcal{Y}, \quad \frac{|\{i \in \mathcal{F}_k : Y_i = c\}|}{|\mathcal{F}_k|} \;\simeq\; \frac{n_c}{n}.$$

Sans stratification, certains plis pourraient ne contenir aucun patient `midl`, rendant l'évaluation locale du risque indéfinie pour cette classe.

Répétitions. Pour réduire la variance de $\widehat{\mathcal{R}}_{\mathrm{CV}}$ liée au tirage particulier des plis, on répète la procédure R fois avec des partitions indépendantes $(\mathcal{F}_k^{(r)})_{k=1}^K$, $r = 1, \dots, R$, et on moyenne :

$$\widehat{\mathcal{R}}_{\text{CV-rep}}(\theta) \;=\; \frac{1}{RK}\sum_{r=1}^{R}\sum_{k=1}^{K} \frac{1}{|\mathcal{F}_k^{(r)}|} \sum_{i \in \mathcal{F}_k^{(r)}} \ell\Big(\hat{f}_\theta^{(-k,r)}(X_i),\, Y_i\Big)\,.$$

Nested Cross-Validation Certains algorithmes (par exemple `cv.glmnet`) possèdent eux-mêmes un hyperparamètre interne λ qu'il faut estimer à partir des données d'entraînement. Pour éviter tout data leakage entre sélection de λ et évaluation du modèle, nous utilisons une CV à deux niveaux :

- Boucle externe : $K \times R = 21$ plis pour évaluer $\widehat{\mathcal{R}}_{\mathrm{CV}}(\theta_{\mathrm{externe}})$, où $\theta_{\mathrm{externe}}$ désigne les hyperparamètres balayés manuellement.
- Boucle interne : à chaque pli externe, on lance une seconde CV (5 ou 10-fold) sur le sous-ensemble d'entraînement pour sélectionner $\hat{\lambda} := \arg\min_\lambda \widehat{\mathcal{R}}^{\,\mathrm{int}}(\lambda)$. Le modèle est ensuite réajusté sur l'ensemble du fold-train avec $\hat{\lambda}$, puis évalué sur le fold-valid externe.

Cette procédure assure que $\hat{\lambda}$ ne voit jamais les observations sur lesquelles le modèle sera évalué, pour éviter toute triche du modèle !

Métrique : balanced accuracy. Avec une prévalence de classe très déséquilibrée, l'accuracy brute pondère implicitement les classes par leur fréquence, ce qui masquerait les performances sur `midl`. On lui préfère alors la balanced accuracy :

$$\mathrm{BA}(\hat{f}) \;=\; \frac{1}{|\mathcal{Y}|}\sum_{c \in \mathcal{Y}} \underbrace{\frac{|\{i : Y_i = c \text{ et } \hat{f}(X_i) = c\}|}{|\{i : Y_i = c\}|}}_{\mathrm{recall}_c(\hat{f})}\,.$$

Cette métrique attribue le même poids aux trois classes, indépendamment de leur prévalence. Cela permet d'ignorer un classifieur qui prédit toujours la classe majoritaire. C'est la métrique qu'on optimise dans toutes les boucles externes et qu'on rapporte dans la section Résultats.

3.8 Méthodologie de fusion de données multiblocs

Notre jeu de données est divisé en deux blocs très distincts : un bloc d'expression génomique, et un bloc CGH d'altérations chromosomiques. Ces deux blocs sont très distincts et surtout déséquilibrés, le bloc GE représentant plus de 90% des features, il a un avantage structurel. Nous présentons ici plusieurs méthodes de fusions essayées au cours du projet.

Premièrement, notons qu'avant ce paragraphe, nous avons fais abstraction total de ce détail en prenant des observations $X_i \in \mathbb{R}^p$ avec p non fixé. On définit alors :

- $X = (X_1, \dots, X_n) \in \mathbb{R}^{n \times p_2}$ les observations du bloc GE, avec $p_2 = 15702$
- $Z = (Z_1, \dots, Z_n) \in \mathbb{R}^{n \times p_C}$ les observations du bloc CGH, avec $p_1 = 1229$

On note $Y = (Y_1, \dots, Y_n) \in \mathcal{Y}^n$ la target

3.8.1 Méthodes naïves

Deux stratégies élémentaires encadrent toutes les autres. La **fusion précoce** (early fusion) concatène les deux blocs en une seule matrice $[\, X \mid Z \,] \in \mathbb{R}^{n \times (p_2 + p_1)}$, puis applique un unique modèle à l'ensemble. Elle est simple mais souffre du déséquilibre dimensionnel décrit en §2 : le bloc GE, qui concentre plus de 90% des variables, domine mécaniquement l'apprentissage et marginalise CGH. La **fusion tardive** (late fusion) entraîne au contraire un modèle indépendant par bloc, puis agrège leurs prédictions, typiquement par une combinaison convexe $\omega\, f_X(X) + (1 - \omega)\, f_Z(Z)$. Elle préserve chaque vue mais introduit un hyperparamètre de pondération ω à régler et ignore par construction toute interaction entre blocs. Le Cooperative Learning, présenté ci-dessous, se lit précisément comme une interpolation continue entre ces deux extrêmes.

3.8.2 Cooperative Learning

Le Cooperative Learning a été introduit par Ding, Tibshirani et collaborateurs (PNAS, 2022) pour l'intégration supervisée de plusieurs vues (multi-omiques notamment). L'idée est de trouver un juste milieu plus intelligent entre fusion précoce et fusion tardive : entraîner les deux modèles f_X et f_Z en même temps, tout en forçant les deux blocs à concorder. On définit alors nos nouveaux classifieurs sous la forme $f(X, Z) = f_X(X) + f_Z(Z)$.

L'article propose, à un hyperparamètre de contrôle $\rho \geqslant 0$ près, de minimiser le risque de population, dans le cas d'une régression avec $\ell = MSE$:

$$\mathcal{R}(f_X, f_Z) = \mathbb{E}\left[\frac{1}{2}(Y - f_X(X) - f_Z(Z))^2 + \frac{\rho}{2}(f_X(X) - f_Z(Z))^2\right]$$

On retrouve en fait notre définition du risque de population $\mathbb{E}(\ell(Y, f(X, Z)))$, auquel s'ajoute une pénalisation « agreement penalty » : on pénalise le modèle si la prédiction issue de la vue X est trop différente de la prédiction issue de la vue Z.

L'hyperparamètre ρ contrôle tout :

- Si $\rho = 0$, aucune pénalité = fusion précoce

- Si $\rho \to +\infty$, l'article montre que la solution est la moyenne des prédictions marginales, ce qui correspond à la fusion tardive

La formulation ci-dessus suppose ℓ = MSE et identifie sans ambiguïté $f_X(X)$ à la prédiction du modèle. En classification multi-classe (dans le cadre des GLM notamment), cette identification est moins immédiate : la prédiction finale est une probabilité $p_c(x) = \text{softmax}_c(X\theta)$ (c.f. section GLM), mais elle n'est qu'une étape après le score linéaire $\eta_c(x) = X^\top \theta_c$. Il faut donc choisir sur quelle couche appliquer la pénalité d'accord. Sur les logits, $X\theta$ est linéaire en θ et la pénalité

$$\mathcal{P}_{\text{accord}}(\theta_x, \theta_z) \;=\; \frac{\rho}{2} \|X\theta_x - Z\theta_z\|_F^2$$

,avec $||.||_F$ la norme de Frobenius, reste une forme quadratique. On identifie donc dans la suite $f_X(X)$ à son logit $X\theta_x$ et $f_Z(Z)$ à $Z\theta_z$, généralisation directe du cas régression où la prédiction est déjà le score linéaire.

Une dernière différence avec le cas binaire de l'article original : notre problème compte $K = 3$ classes. La régression logistique multinomiale (§3.3) requiert non plus un vecteur de coefficients mais une matrice $\beta \in \mathbb{R}^{p \times K}$, avec une colonne β_c par classe. Le cadre cooperative s'adapte naturellement en remplaçant les vecteurs θ_x, θ_z par deux matrices, une par vue.

Nous disposons désormais de deux matrices de paramètres : $\theta_x \in \mathbb{R}^{p_x \times 3}$ pour la vue X, et $\theta_z \in \mathbb{R}^{p_z \times 3}$ pour la vue Z. Le score pré-softmax pour une observation i et une classe c issu de la première vue est $X_i^T \theta_{x,c}$, et celui issu de la seconde vue est $Z_i^T \theta_{z,c}$.

Le score combiné total est :

$$\eta_{i,c} = X_i^T \theta_{x,c} + Z_i^T \theta_{z,c}$$

La fonction objectif globale $J(\theta_x, \theta_z)$ que notre modèle cherche à minimiser (le risque empirique pénalisé) se décompose alors en trois termes :

$$J(\theta_x, \theta_z) = \hat{R}_{CE}(\theta_x, \theta_z) + \mathcal{P}_{accord}(\theta_x, \theta_z) + \mathcal{P}_{Elastic-Net}(\theta_x, \theta_z)$$

Le premier terme est l'entropie croisée, conformément aux sections précédentes.
Le terme $\mathcal{P}_{accord}(\theta_x, \theta_z)$ est une pénalité d'accord, qui force les signaux entre les deux blocs à s'aligner, en minimiser la norme de Frobenius $||.||_F$ de la distance entre les matrices de scores

$$\mathcal{P}_{accord}(\theta_x, \theta_z) = \frac{\rho}{2} ||X\theta_x - Z\theta_z||_F^2$$

Enfin, une pénalisation Elastic-Net $\mathcal{P}_{net}$ qui régularise le modèle :

$$\mathcal{P}_{Elastic-Net}(\theta_x, \theta_z) = \lambda \left[\alpha (||\theta_x||_1 + ||\theta_z||_1) + \frac{1-\alpha}{2} (||\theta_x||_2^2 + ||\theta_z||_2^2) \right]$$

avec α le paramètre de mélange, λ le paramètre d'intensité.

Données augmentées Une fois la fonction objectif $J(\theta_x, \theta_z)$ formellement définie, la difficulté réside dans son optimisation conjointe. Plutôt que de développer un solveur d'optimisation sur mesure, l'article propose une méthode approche différente.

Dans le cadre des GLM, l'optimisation s'effectue classiquement par la méthode de Newton des moindres carrés repondérés, ou de manière équivalente la méthode de Newton-Raphson. À chaque itération, l'algorithme simeqime la log-vraisemblance par un problème des moindres carrés pondérés, faisant intervenir une matrice de poids diagonale W et un vecteur de variables ajustées z.

L'article démontre que la pénalité d'accord peut être parfaitement intégrée à ce processus en construisant, à chaque itération, une matrice augmentée de données $\tilde{X}$ et un vecteur augmenté cible $\tilde{z}$ définis par l'empilement suivant :

$$\tilde{X} = \begin{pmatrix} W^{1/2}X & W^{1/2}Z \\ -\sqrt{\rho}X & \sqrt{\rho}Z \end{pmatrix}, \quad \tilde{z} = \begin{pmatrix} z \\ 0 \end{pmatrix}, \quad \tilde{\beta} = \begin{pmatrix} \theta_x \\ \theta_z \end{pmatrix}$$

Ainsi, le problème d'optimisation du cooperative learning est équivalent à résoudre un problème elastic-net standard pour le jeu de données augmenté $(\tilde{X}, \tilde{z})$.

3.9 RGCCA et SGCCA

L'analyse canonique généralisée régularisée (RGCCA, Tenenhaus & Tenenhaus 2011 ; Tenenhaus et al. 2017) est un cadre unifié d'analyse multibloc qui cherche des composantes, une par bloc, maximisant un critère d'association entre blocs sous contraintes de norme. Sa variante parcimonieuse, la SGCCA (Sparse Generalized Canonical Correlation Analysis), ajoute une pénalité ℓ^1 qui force la sélection de variables au sein de chaque bloc, ce qui la rend particulièrement adaptée au régime $p \ggg n$ de notre projet. En incluant le bloc des étiquettes parmi les blocs analysés, la SGCCA fournit une projection supervisée : c'est exactement le levier identifié comme nécessaire à l'issue de l'analyse exploratoire (§2), où la PCA non supervisée échouait à séparer les classes.

Formalisme RGCCA-SGCCA

On suppose observer des données multiblocs sous forme de J matrices de données $X_1, \ldots, X_J$ où $X_j \in \mathbb{R}^{n \times p_j}$ est un bloc de p_j variables observées sur le même ensemble de n individus. On supposera les variables centrées.

L'objectif est de trouver un vecteur de poids de bloc $a_j = (a_{j,1}, \ldots, a_{j,p_j})^T$ tel que :

- $y_j = X_j a_j$ expliquent bien leur propre bloc
- Les composantes de blocs connectées sont fortement corrélées.

Cela revient à résoudre le problème d'optimisation :

$$\max_{a_1,\ldots,a_J} \sum_{j,k=1}^{J} c_{j,k} g(\mathrm{cov}(X_j a_j, X_k a_k)) \tag{1}$$

où $a_j \in \Omega_j$ et :

- Ω_j est un compact à définir
- g une fonction convexe continuement différentiable
- Matrice de connexion $C = (c_{j,k})_{1 \leqslant j,k \leqslant J}$ est une matrice symétrique décrivant le réseau de connexions entre les blocs. Habituellement, $c_{j,k} = 1$ si les blocs sont connectés, 0 sinon.

La résolution du problème (1) donne un ensemble de J vecteurs $a_j^{(1)}$ où $a_j^{(1)}$ quantifie la contribution des variables du bloc X_j à la construction des composantes de blocs $y_j^{(1)} = X_j a_j^{(1)}$.

SGCCA : Sparse Generalized Canonical Correlation Analysis

Objectif : identifier au sein de chaque bloc les sous-ensembles de variables significatives qui sont les plus pertinentes pour les relations entre les blocs. Le but est donc de **sélectionner** des variables (très important au vu de la dimensionnalité de notre projet). À cette fin, on utilise la pénalisation ℓ^1 en choisissant :

$$\Omega_j = \left\{ a_j \in \mathbb{R}^{p_j} : ||a_j||_2^2 \leqslant 1 \text{ et } ||a_j||_1 \leqslant s_j \right\}$$

où $s_j > 0$ est un hyperparamètre. Plus s_j est petit, plus le degré de parcimonie (« sparsité ») de a_j est grand. La géométrie de la boule ℓ^1 explique le mécanisme : le simplexe $\{a \in \mathbb{R}^{p_j} : \|a\|_1 \leqslant s_j\}$ est un polytope dont les sommets sont les axes canoniques mis à l'échelle. Plus s_j est petit, plus ces sommets sont rapprochés de l'origine et plus la solution optimale du problème (1), qui maximise une fonction lisse sous contrainte, tend à venir s'aligner sur une face de basse dimension, c'est-à-dire à annuler un grand nombre de coordonnées de a_j.

Pour garantir un domaine non vide, l'article impose $\frac{1}{\sqrt{p_j}} \leqslant s_j \leqslant 1$. La borne inférieure découle de l'inégalité de Cauchy-Schwarz : pour tout a_j vérifiant $\|a_j\|_2 \leqslant 1$, on a $\|a_j\|_1 \leqslant \sqrt{p_j}\,\|a_j\|_2 \leqslant \sqrt{p_j}$, et le rapport $\|a_j\|_1/\|a_j\|_2$ est minoré par 1 (atteint sur les sommets canoniques). Choisir $s_j < 1/\sqrt{p_j}$ rendrait Ω_j vide.

Algorithme

Reprenons le problème (1). On note f la fonction objectif du problème. Soit $a = (a_1, \ldots, a_J)$. On suppose que $\nabla_j f(a) \neq 0$ (hypothèse pas déconnante, car $\nabla_j f(a) = 0$ caractérise précisemment les extrêmas de f, et donc si égalité est satisfaite, alors pas besoin de passer par l'algorithme...).

Puisque f est C^1 multi-convexe, on a, pour $\tilde{a}_j \in \Omega_j$:

$$f(a_1, \ldots, a_{j-1}, \tilde{a}_j, a_{j+1}, \ldots, a_J) \geqslant f(a) + \nabla_j f(a)^T (\tilde{a}_j - a_j)$$

On cherche donc $\hat{a}_j = \underset{\tilde{a}_j \in \Omega_j}{\operatorname{argmax}} \nabla_j f(a)^T (\tilde{a}_j - a_j) = r_j(a)$.

Algorithme 1 Tenenhaus et al. 2017.

Algorithm 1: Block relaxation pour le problème (1)

Input: matrices $X_1, \ldots, X_J$, matrice de connexion C, fonction g, contraintes Ω_j, tolérance $\varepsilon > 0$

Output: $\mathbf{a}_1^*, \ldots, \mathbf{a}_J^*$ solution approchée de (1)

Initialisation : choisir $\mathbf{a}_j^0 \in \Omega_j$ au hasard pour $j = 1, \ldots, J$;
$s \leftarrow 0$;
repeat
  for $j = 1$ **to** J **do**
    $\mathbf{a}_j^{s+1} \leftarrow r_j\left(\mathbf{a}_1^{s+1}, \ldots, \mathbf{a}_{j-1}^{s+1}, \mathbf{a}_j^s, \ldots, \mathbf{a}_J^s\right)$;
  $s \leftarrow s + 1$;
until $\left| f(\mathbf{a}_1^{s+1}, \ldots, \mathbf{a}_J^{s+1}) - f(\mathbf{a}_1^s, \ldots, \mathbf{a}_J^s) \right| < \varepsilon$;

À chaque itération, l'algorithme met à jour les blocs l'un après l'autre (cyclic block coordinate ascent, De Leeuw 1994) en remplaçant chaque $\mathbf{a}_j$ par le maximiseur $r_j(\mathbf{a})$ de la minorante linéaire de f en $\mathbf{a}_j$. La fonction f étant continûment différentiable et multi-convexe sur le compact $\Omega_1 \times \cdots \times \Omega_J$, la suite $(f(\mathbf{a}^s))_{s \geqslant 0}$ est croissante et majorée, donc convergente ; le point de convergence est un point stationnaire de f (Tenenhaus et al. 2017, théorème 1).

On trouve finalement,

$$r_j(a) = \frac{S(\nabla_j f(a), \lambda_j)}{||S(\nabla_j f(a), \lambda_j)||_2}$$

Où $\mathcal{S}$ est un opérateur de soft-tresholding, λ_j est un seuil.

Choix du seuil λ_j. Le paramètre λ_j joue le rôle de multiplicateur de Lagrange associé à la contrainte $\|a_j\|_1 \leqslant s_j$. Sa valeur est déterminée par la situation de cette contrainte à l'optimum :

$$\lambda_j \;=\; \begin{cases} 0 & \text{si } \dfrac{\|\nabla_j f(\mathbf{a})\|_1}{\|\nabla_j f(\mathbf{a})\|_2} \;\leqslant\; s_j, \\ \\ \text{tel que } \|r_j(\mathbf{a})\|_1 = s_j & \text{sinon.} \end{cases}$$

On a alors, par le KKT : $\lambda_j \left(\dfrac{||\nabla_j f(a)||_1}{||\nabla_j f(a)||_2} - s_j \right) = 0$

- Si la contrainte ℓ^1 est inactive, alors aucun seuillage n'est nécessaire, et $\lambda_j = 0$
- Si la contrainte ℓ^1 est active, alors on cherche le plus petit λ_j saturant la contrainte $||r_j(a)||_1 = s_j$. Ce λ_j est unique et se calcule soit par recherche binaire, soit par l'approche analytique de Gloaguen et al. (2017).[11]

Application à notre problème

On dispose de trois blocs :

- $X = X_1$: $p_1 = 15702$ gènes du bloc (GE)
- $Z = X_2$: $p_2 = 1229$ segments de chromosomes du bloc (CGH)
- X_3 : $p_3 = 1$ variables indicatrices de localisation (y_{train})

L'idée est de réduire les blocs X_j en composantes de blocs $y_j = X_j a_j$.
On commence par standardiser :

$$z_{i,k} = \frac{x_{i,k} - \mu_k}{\sigma_k}$$

pour chaque blocs. Puisqu'on a standardiser, la variance de chaque variable vaut 1, et donc $\text{Var}(X_j)$ contient une diagonale de 1. Ainsi, la somme des valeurs propres de la matrice de covariances vaut p_j $\longrightarrow$ une inertie de p_j, ce qui créer un déséquilibre. On met alors à l'échelle :

$$\tilde{X_j} = \frac{1}{\sqrt{p_j}} X_j$$

De sorte que $\text{Tr}(\tilde{X_j}^T X_j) = 1$

En pratique, le prétraitement appliqué en amont de la SGCCA est le suivant : imputation des valeurs manquantes par la médiane calculée sur l'ensemble d'entraînement (puis reportée sur le test, sans fuite d'information), standardisation z-score variable par variable, et mise à l'échelle par bloc $\tilde{X}_j = X_j/\sqrt{p_j}$ décrite ci-dessus afin d'égaliser l'inertie des blocs. Les composantes de bloc issues de la SGCCA alimentent ensuite une analyse discriminante linéaire (LDA) : la LDA opère ainsi dans l'espace réduit à quelques composantes supervisées, et non sur les $p \simeq 17\,000$ variables initiales, ce qui contourne la non-inversibilité de la matrice de covariance en régime $p \ggg n$. Les performances correspondantes sont rapportées dans la section Résultats.

[11] Nous n'avons pas creusé le détail de ce calcul de seuil et avons utilisé l'implémentation du package `RGCCA` telle quelle.

4 Résultats

4.1 Sélection de variables

SGCCA

L'application de la SGCCA à notre modèle a permis de retenir :

- 68 variables du bloc GE
- 11 variables du bloc CGH

Ces variables ont été sélectionnées comme celles dont la coordonnée correspondante dans les vecteurs de poids $a_{\mathrm{GE}}^{(1)}$ et $a_{\mathrm{CGH}}^{(1)}$ est non nulle à l'issue de l'algorithme.

On rappelle le problème d'optimisation : Soit $J = 3$ blocs : $X_1 = X_{\mathrm{GE}} \in \mathbb{R}^{n \times p_1}$ ($p_1 = 15\,702$), $X_2 = X_{\mathrm{CGH}} \in \mathbb{R}^{n \times p_2}$ ($p_2 = 1\,229$), et $X_3 = Y \in \{0,1\}^{n \times 3}$.
La matrice de connexion est :

$$C = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix} \begin{matrix} (\mathrm{GE}) \\ (\mathrm{CGH}) \\ (y) \end{matrix}$$

Ce choix encode l'hypothèse suivante : on ne cherche pas à corréler directement GE et CGH entre eux, mais bien à corréler chacun des deux blocs moléculaires au bloc des labels. Le bloc y joue ici le rôle de pivot supervisé : c'est lui qui contraint la SGCCA à retenir des composantes discriminantes, et non simplement des composantes de forte variance comme le ferait une PCA classique.

Avec $g(x) = x^2$ les composantes de blocs $y_j = X_j a_j$, le critère SGCCA s'écrit :

$$\max_{a_1,\, a_2,\, a_3} \ \mathrm{cov}\big(X_1 a_1,\, X_3 a_3\big)^2 \ + \ \mathrm{cov}\big(X_2 a_2,\, X_3 a_3\big)^2$$

sous les contraintes

$$\begin{cases} \|a_1\|_2^2 \leqslant 1 \ \text{ et } \ \|a_1\|_1 \leqslant s_1 \\ \|a_2\|_2^2 \leqslant 1 \ \text{ et } \ \|a_2\|_1 \leqslant s_2 \\ \|a_3\|_2^2 \leqslant 1 \end{cases}$$

où $s_1 = 0{,}051$ et $s_2 = 0{,}067$.

Les hyperparamètres de sparsité (s_1, s_2) ont été choisis par cross-validation sur une grille de huit configurations, en suivant le protocole recommandé dans la documentation du package `RGCCA`. La configuration retenue est celle qui maximise la balanced accuracy moyenne sur les folds de validation, soit `Set 8` avec La valeur du critère objectif au point optimal vaut

$$\sum_{j,k} c_{jk}\, g\big(\mathrm{cov}(X_j a_j, X_k a_k)\big) = 0.0042.$$

Ce niveau, faible en valeur absolue, mérite quelques mots d'interprétation. Les variables ayant été préalablement standardisées puis mises à l'échelle par $1/\sqrt{p_j}$, l'inertie totale de chaque bloc a été normalisée à 1, ce qui contracte mécaniquement la magnitude des covariances entre composantes. Une valeur de l'ordre de 10^{-3} ne reflète donc pas une faiblesse du signal extrait, mais bien l'échelle imposée par la normalisation des blocs. Ce qui importe ici, c'est moins la valeur brute du critère que la stabilité et la qualité discriminante des composantes $X_j a_j$ obtenues, que nous évaluons en aval par un classifieur LDA et que nous validons par un test de permutation.

Cooperative Learning

On rappelle le problème d'optimisation du cooperative learning appliqué aux GLM. Avec $\theta_x \in \mathbb{R}^{p_x \times 3}$ et $\theta_z \in \mathbb{R}^{p_z \times 3}$ les deux matrices de paramètres (une colonne par classe $c \in \{1, \dots, 3\}$), le score pré-softmax pour l'observation i et la classe c s'écrit

$$\eta_{i,c} \;=\; X_i^\top \theta_{x,c} \;+\; Z_i^\top \theta_{z,c},$$

et le problème de minimisation conjoint s'écrit

$$\min_{\theta_x, \theta_z} \underbrace{\hat{\mathcal{R}}_{CE}(\theta_x, \theta_z)}_{\text{cross-entropie}} + \underbrace{\frac{\rho}{2} \left\| X\theta_x - Z\theta_z \right\|_F^2}_{\mathcal{P}_{\text{accord}}(\theta_x, \theta_z)} + \underbrace{\lambda \Big[\alpha(\|\theta_x\|_1 + \|\theta_z\|_1) + \tfrac{1-\alpha}{2}(\|\theta_x\|_2^2 + \|\theta_z\|_2^2) \Big]}_{\mathcal{P}_{\text{Elastic-Net}}(\theta_x, \theta_z)}.$$

Trois hyperparamètres contrôlent ce problème :

- $\rho \geqslant 0$: mélange des blocs.
- $\lambda > 0$: intensité globale de la pénalisation.
- $\alpha \in [0, 1]$: mélange Lasso-Ridge.

On procède alors à une cross-validation sur ρ pour trouver le paramètre optimal. On rappelle que pour le cas de la régression logistique, l'entraînement passe par la méthode de Newton, qui linéarise à chaque itération la log-vraisemblance pour la ramener à un problème de moindres carrés pondérés. Sur notre jeu de données, en utilisant la librairie `multiview` qui repose en interne sur `glmnet`, on observe que pour toute valeur $\rho > 0$ l'algorithme diverge systématiquement, retournant l'avertissement `glmnet.fit : algorithm did not converge` à chaque fold de la validation croisée. On peut essayer de comprendre cette divergence :

- les poids de Newton $w_i = \hat{p}_i(1 - \hat{p}_i)$ tendent vers 0 sur la classe rare, rendant $X^\top W X$ quasi-singulière (on sort donc des hypothèses standards de la convergence de l'algorithme de Newton)
- l'agrément ajoute $\rho\, X^\top X$ à la matrice de Gram et amplifie d'un facteur $(1 + \rho)$ des corrélations intra-bloc déjà extrêmes

On peut ajouter une lecture plus structurelle de cette instabilité : la pénalité ℓ^1 pousse à annuler des coefficients, tandis que la pénalité d'accord $\mathcal{P}_{\text{accord}}$ pousse au contraire à coupler les deux vues en alignant leurs scores. Lorsque les deux blocs ne partagent pas de signal commun fort, ces deux forces tirent vers des configurations contradictoires (sparse d'un côté, fortement couplée de l'autre), ce qui peut contribuer à la mauvaise conditionnement du problème augmenté et à la non-convergence observée.

Empiriquement, le package `multiview` échoue à converger (`glmnet did not converge`, >100 itérations) pour tout $\rho \in \{0{,}1\,;\, 0{,}5\,;\, 1\,;\, 2\,;\, 5\}$. Seul $\rho = 0$ donne un résultat exploitable, et la méthode dégénère alors en une régression Lasso One-vs-Rest (détaillé dans le point suivant).

Sur ce constat, nous avons tenté deux interprétations. La première est conceptuelle : les blocs GE et CGH sont structurellement en désaccord, soit parce qu'ils sont complémentaires, soit parce que le bloc CGH se trompe systématiquement et que tout terme d'agrément ne fait que dégrader la prédiction. Cette dernière hypothèse devrait cependant se refléter dans la sélection de variables opérée par la SGCCA, ce qui n'est pas le cas puisque celle-ci retient 11 variables CGH. La seconde interprétation est que le verrou est purement algorithmique : c'est la mise à jour de Newton de `glmnet` qui ne supporte pas la perturbation introduite par la matrice augmentée à $\rho > 0$, indépendamment du fait

qu'un ρ optimal théorique existe ou non. Nous ne sommes pas parvenus à lever ce problème pour la formulation One-vs-Rest. En revanche, la formulation multinomiale ne présente pas cette divergence : elle converge et fournit $\rho = 0{,}10$, mais au prix d'une balanced accuracy nettement plus basse (voir section suivante).

Régression logistique multinomiale : une question d'algorithme

Ce qui précède soulève une question plus générale, qui a structuré une partie de notre travail expérimental : la façon dont la régression logistique multiclasse est implémentée en pratique a un impact direct sur les résultats obtenus, à formulation mathématique pourtant identique.

Le langage `R` offre deux modes principaux pour traiter un problème multiclasses à travers la librairie `glmnet` :

- Résolution multinomiale: optimise directement la log-vraisemblance multinomiale sous contrainte softmax, c'est-à-dire celle qui apparaît plus tôt dans ce rapport.
- Résolution OvR (One vs Rest) : décompose le problème en K (K = nombre de classes) régressions logistiques binaires indépendantes, chacune cherchant à séparer une classe contre l'union des deux autres, et la prédiction finale est obtenue par $\hat{y} = \underset{k}{\text{argmax}}\ \hat{P}_k(x)$ où chaque $\hat{P}_k$ est calibrée séparément.

Bien que ces deux formulations soient asymptotiquement équivalentes pour un classifieur bayésien, elles n'ont pas l'air de se comporter de la même manière pour $p \ggg n$, surtout avec des classes déséquilibrés.
Il semble que la différence vient du couplage des probabilités. En multinomial, la contrainte $\sum_c P_c(x) = 1$ est imposée par construction via softmax, ce qui couple les coefficients β_c entre eux : si la classe midl est largement minoritaire, le softmax tend à écraser sa probabilité au profit de classes majoritaires, et donc la pénalité ℓ^1 annule β_{midl}.

Au contraire, en OvR, chaque classifieur binaire est entraîné indépendamment. De fait, si la classe midl est trop rare pour qu'un signal soit détecté, ses coefficients β s'annulent également, mais l'intercept reste calibré sur la prévalence empirique, et la probabilité prédite (= l'intercept) $\hat{P}_{\text{midl}} \simeq 0.20$ pour tous les patients, indépendamments des features $\longrightarrow$ probabilité constante qui agit comme un « filet de sauvetage », et l'argmax peut alors repêcher midl de temps en temps.

Quantitativement, sur notre jeu de données, cette différence se traduit par un écart important de balanced accuracy, et évidemment par un recall midl de 2/3 en OvR contre 0/3 en multinomial.

À noter : avec la régression logistique multinomiale classique, l'algorithme de Newton ne diverge pas dans la résolution du problème d'optimisation du cooperative learning, et on obtient $\rho^* = 0.10$, avec une accuracy très proche d'une régression logistique multinomiale classique.

De manière générale, les performances obtenues avec OvR sont proches de la première méthode avec sélection de variables par SGCCA. Toutefois, cette dernière semble plus robuste et plus pertinente.

Groupped vs Ungroupped : La régression multinomiale pénalisée admet (encore..) deux variantes selon la façon dont la pénalité ℓ^1 agit sur la matrice $\beta \in \mathbb{R}^{p \times K}$:

- Groupped : $\lambda \sum_{j=1}^{p} \|\beta_{j,\cdot}\|_2 \longrightarrow$ norme ℓ^2 ligne par ligne, puis somme ℓ^1 sur les lignes. La sparsité agit au niveau variable : un gène est sélectionné pour les K classes simultanément, ou écarté pour toutes. C'est intéressant pour l'interprétation biologique.

- Ungroupped : $\lambda \sum_{j=1}^{p} \sum_{c=1}^{K} |\beta_{j,c}| \longrightarrow$ chaque coefficient (j, c) est pénalisé indépendamment. Chaque classe sélectionne ses propres variables.

On obtient des résultats différents selon la méthode utilisée.

Résultats finaux et comparaison

Quatre pipelines sparses supervisés ont été évalués sur le même protocole (CV stratifiée $7 \times 3 = 21$ plis, balanced accuracy, test set tenu à part). L'objectif est ici moins d'élire un « meilleur » classifieur que de comprendre quelles propriétés méthodologiques pilotent réellement la performance dans le régime $p \ggg n$ avec classe rare.

Tableau de synthèse :

Méthode	**CV bal_acc**	**Test bal_acc**	**midl test**	$\mathbf{N_{vars}}$
Multinomial Lasso grouped	$0{,}784 \pm 0{,}096$	0,773	0/3	27 GE
Multinomial Lasso ungrouped	$0{,}838 \pm 0{,}123$	0,773	0/3	27 GE
Cooperative Lasso OvR ($\rho = 0$)	$0{,}833 \pm 0{,}129$	0,924	2/3	42 GE
SGCCA + LDA	$0{,}829 \pm 0{,}133$	0,924	2/3	68 GE + 11 CGH

Tab. 1: Comparaison des approches évaluées sur le même protocole (CV 7-fold × 3, test $n = 14$). La colonne midl rapporte le nombre de vrais midl correctement reclassés sur les 3 patients midl du test.

Premièrement, sur le test, deux pipelines seulement récupèrent midl (2/3), ce sont précisément ceux qui exploitent une formulation One-vs-Rest (Cooperative OvR) ou une régression dans un espace latent supervisé (SGCCA + LDA).

Deuxièmement, on remarque que passer de la pénalité ℓ^1 groupée à la pénalité ℓ^1 par coefficient (`type.multinomial : "grouped" → "ungrouped"`) fait gagner +0,054 en CV au seul prix d'un changement d'argument. Par contre, on ne gagne pas de performance sur le test, ce qui n'est pas trop étonnant.
Nous avons également testé une sélection de variables non-supervisée (PCA Sparse dans les folds d'une régression logistique), mais avec des résultats désastreux que nous ne présentons pas ici.

NB : Comme nous l'avons mentionné précédemment, nous avons également testé la méthode cooperative en résolution multinomiale. Nous avions obtenu $\rho^* = 0.10$ mais les mêmes résultats (simeqimativement) que les deux multinomiales.

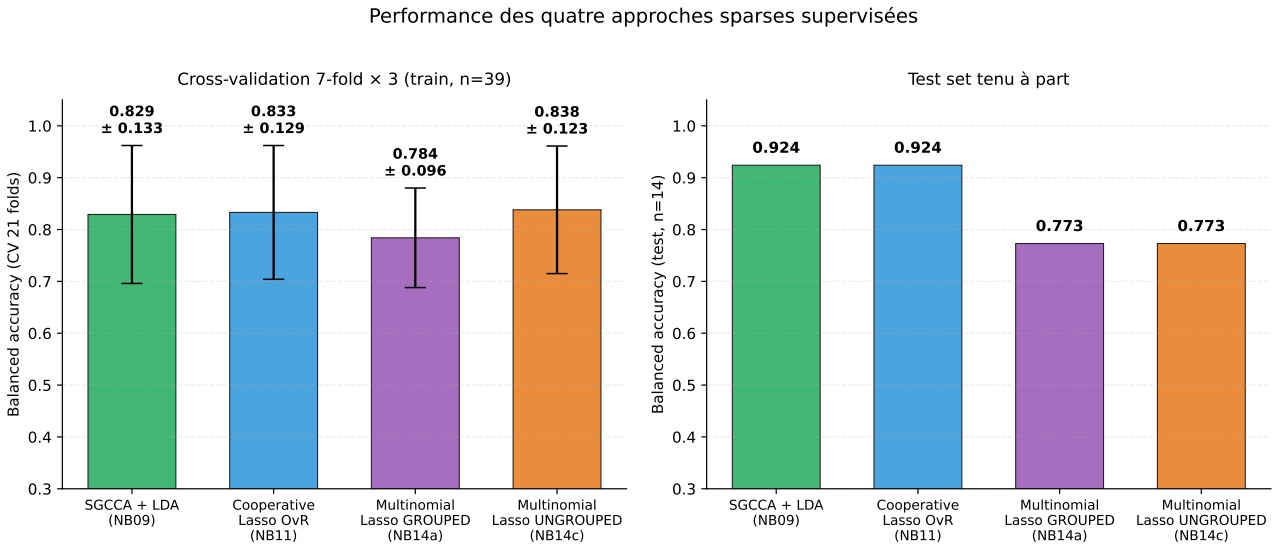


Fig. 7: Performance comparée des quatre approches sparses supervisées. À gauche, balanced accuracy en cross-validation stratifiée (7-fold × 3 = 21 plis, $n_{\text{train}} = 39$), avec barres d'écart-type. À droite, balanced accuracy sur le test tenu à part ($n_{\text{test}} = 14$). En CV, trois méthodes se partagent le plafond à $\simeq 0{,}83$; sur le test, seules SGCCA + LDA et Cooperative OvR atteignent 0,924.

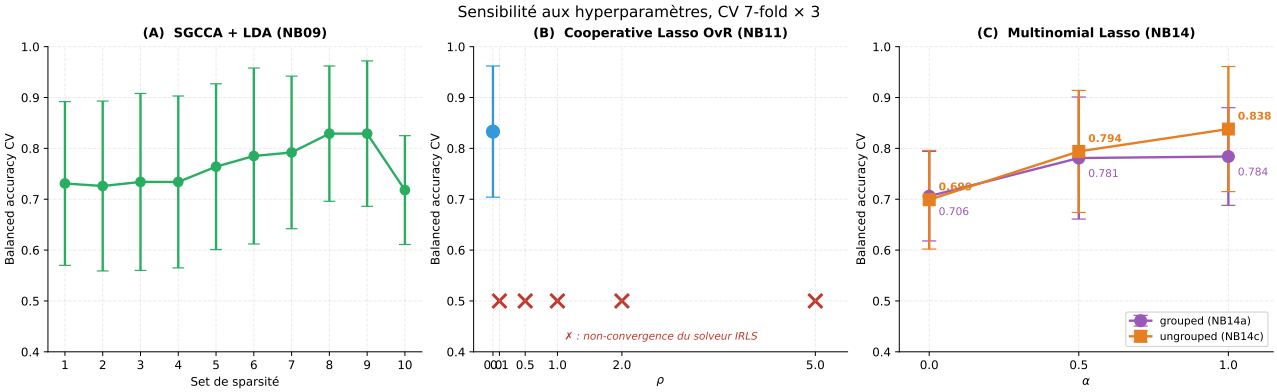


Fig. 8: Sensibilité aux hyperparamètres. **(A)** SGCCA + LDA : balayage de la sparsité sur les dix configurations testées par `rgcca_cv`, optimum aux Sets 8 et 9. **(B)** Cooperative Lasso OvR : sweep de la pénalité d'accord ρ ; les croix rouges marquent les valeurs pour lesquelles le solveur de `multiview` ne converge pas, ne laissant exploitable que $\rho = 0$. **(C)** Multinomial Lasso : sweep de α avec comparaison `grouped` (violet) vs `ungrouped` (orange) ; à $\alpha = 1$, le changement de `type.multinomial` fait gagner +0,054 en balanced accuracy CV.

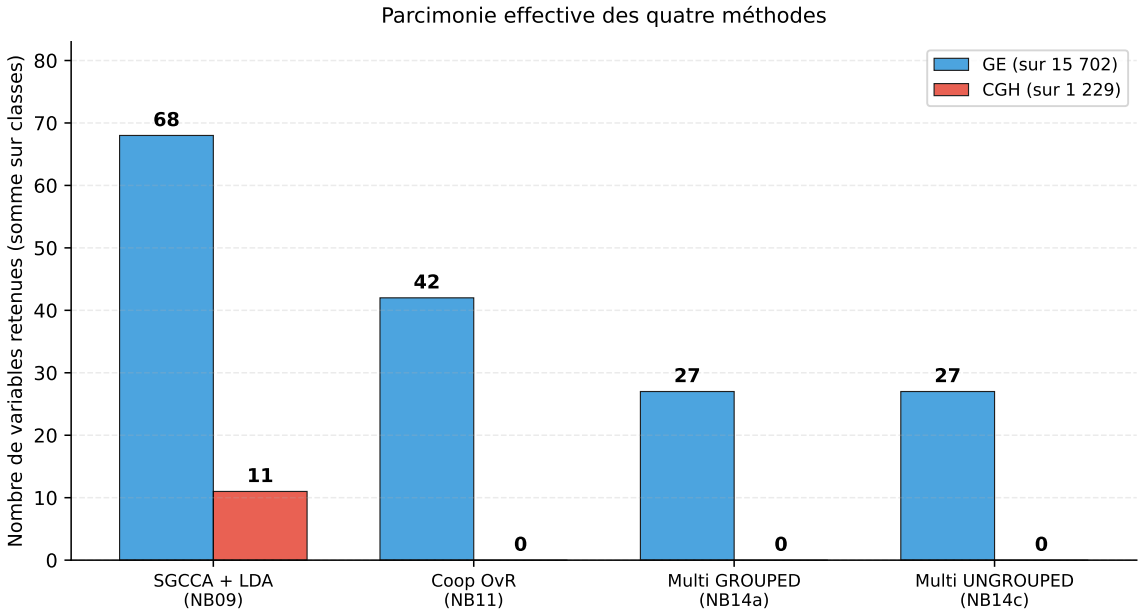


Fig. 9: Sélection de variables des méthodes

Toutes les méthodes data-driven excluent spontanément l'intégralité du bloc CGH ; seule SGCCA en conserve 11 variables, et ce uniquement parce que la contrainte s_{CGH} est imposée structurellement. Le cooperative ne sélectionne pas dans CGH puisque $\rho = 0$...

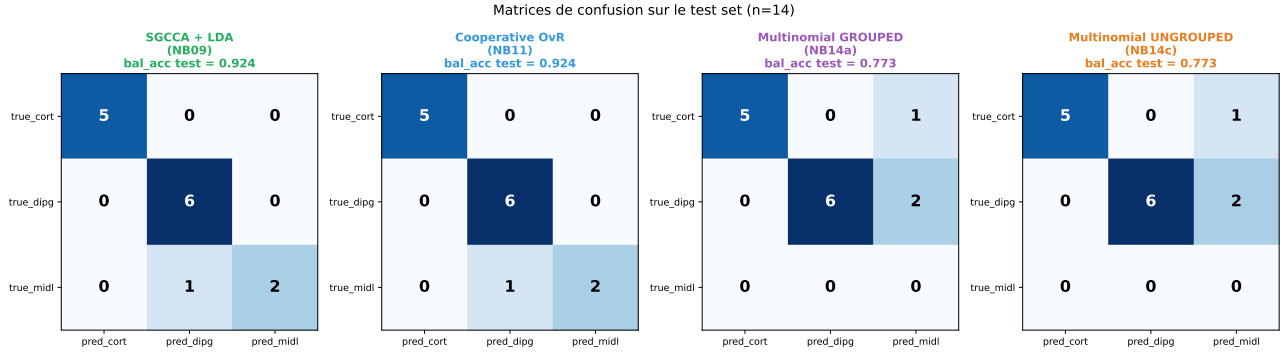


Fig. 10: Différentes matrices de confusion

Les quatre méthodes affichent un comportement quasi-identique sur cort et dipg ; la différence se joue exclusivement sur la classe midl ($n = 3$ patients test). SGCCA et Cooperative OvR récupèrent 2/3 midl, les deux variantes du multinomial Lasso 0/3, redirigées systématiquement vers dipg.

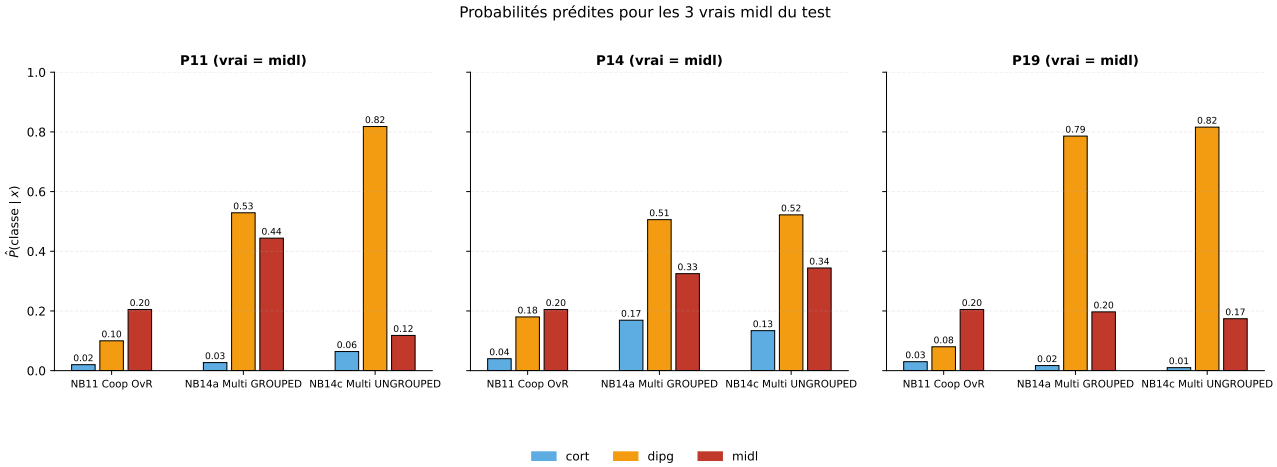


Fig. 11: Repêchage de midl par la régression OvR

La Cooperative OvR prédit systématiquement $\hat{P}_{\text{midl}} = \sigma(\log(8/31)) \simeq 0{,}205$, un plancher qui suffit à l'emporter par argmax pour les exemples de patient P11 et P19. Les deux variantes du multinomial Lasso, contraintes par $\sum_c P_c = 1$, voient cette masse aspirée par dipg.

5 Stabilité de nos résultats

L'analyse de robustesse présentée ici mobilise trois outils méthodologiques classiques : le ré-échantillonnage *bootstrap* [Efr79] et la sélection par stabilité [MB10] pour évaluer la stabilité de la sélection de variables, et le test de permutation appliqué à la performance d'un classifieur [OG10] pour en établir la significativité statistique.

5.1 Stabilité de la sélection de variables

En régime $p \ggg n$, l'identité exacte des variables retenues est intrinsèquement instable : de légères perturbations de l'échantillon d'entraînement peuvent modifier la liste des gènes sélectionnés. Nous avons quantifié ce phénomène par un ré-échantillonnage *bootstrap* [Efr79] : pour chaque sous-échantillon, le modèle parcimonieux est ré-estimé et l'on relève la fréquence de sélection de chaque variable (figure 12). Deux constats se dégagent :

- un **noyau de gènes** du bloc GE est sélectionné avec une fréquence élevée à travers les ré-échantillons ; ce sont les variables sur lesquelles nos conclusions reposent le plus solidement ;
- l'exclusion quasi systématique du bloc CGH est, elle, remarquablement **stable** : aucune méthode data-driven ne lui accorde de poids, ce qui constitue un résultat robuste et non l'artefact d'un découpage particulier.

La fréquence de sélection ainsi mesurée joue le rôle de la probabilité de sélection $\hat{\Pi}_k$ de la sélection par stabilité [MB10] : ne retenir que les variables dont la fréquence dépasse un seuil $\pi_{\text{thr}} \in (\frac{1}{2}, 1)$ contrôle le nombre attendu de faux positifs, au prix d'une légère perte de sensibilité.

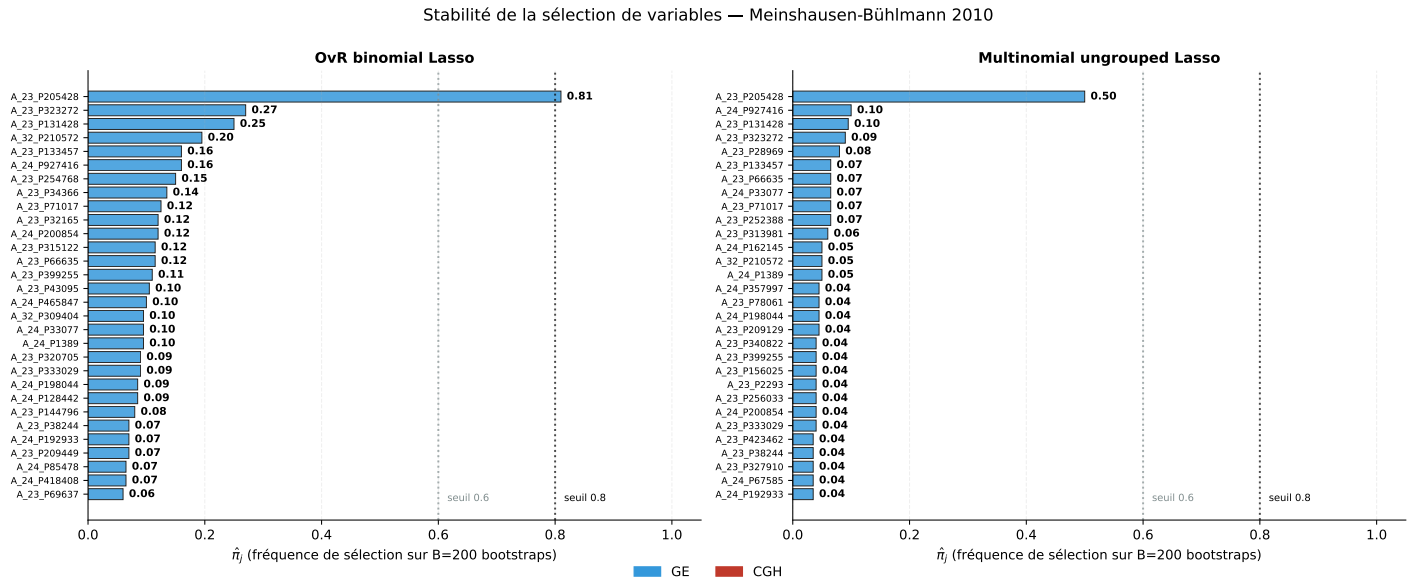


Fig. 12: Fréquence de sélection des variables sur B ré-échantillons bootstrap. Un petit noyau de gènes GE est sélectionné de façon stable, tandis que la grande majorité des variables ne l'est que sporadiquement.

La même procédure de ré-échantillonnage, appliquée cette fois au modèle SGCCA, donne une image sensiblement différente (figure 13). Là où le Lasso One-vs-Rest concentre la stabilité sur un

noyau éduit, autour d'un gène GE dominant (`A_23_P205428`, $\hat{\Pi}_k \approx 0,81$) nettement détaché du reste, SGCCA répartit la sélection sur une longue liste de gènes dont la fréquence décroît graduellement (de $\hat{\Pi}_k \approx 0,74$ à $0,30$), sans rupture franche : seules deux ou trois variables franchissent le seuil $\pi_{\text{thr}} = 0,6$, et la large bande intermédiaire $(0,3 - 0,6)$ change de composition d'un ré-échantillon à l'autre. Le bloc CGH, enfin, n'y contribue qu'un bruit diffus de basse fréquence ($\hat{\Pi}_k \leq 0,11$).

Cette diffusion des poids est cohérente avec le mauvais conditionnement du problème augmenté évoqué plus haut, où les pénalités ℓ^1 et d'accord tirent vers des configurations contradictoires : faute de signal commun fort entre les deux blocs, SGCCA ne parvient pas à stabiliser un sous-ensemble reproductible de variables. À performance comparable, la régression logistique pénalisée offre donc une sélection plus stable et plus interprétable, et c'est cet écart de stabilité qui motive le choix de la LogReg comme modèle de référence.

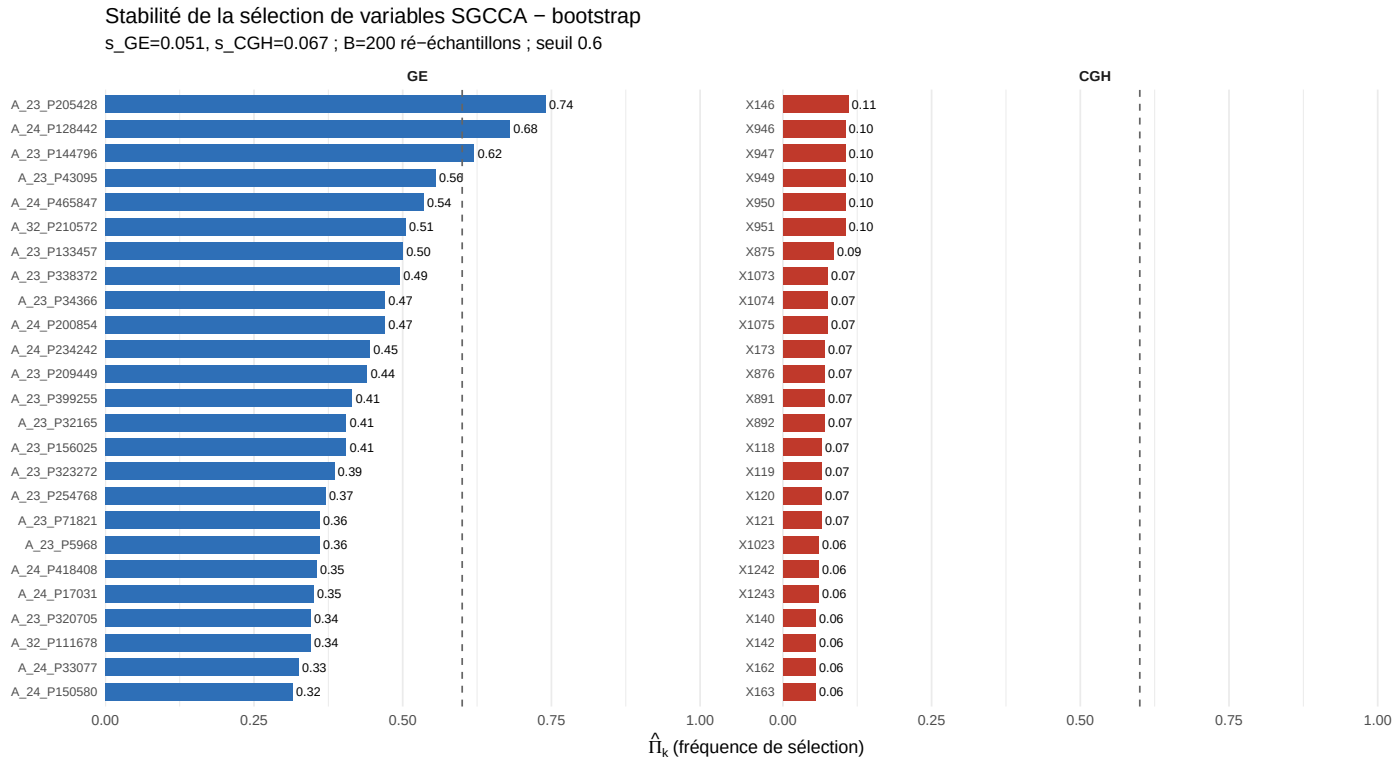


Fig. 13: Fréquence de sélection des variables sous SGCCA, sur $B = 200$ ré-échantillons bootstrap ($s_{GE} = 0,051, s_{CGH} = 0,067$, seuil $0,6$). Comparée à la figure 12, la sélection y est nettement plus diffuse : aucun noyau franc ne se détache et le bloc CGH demeure non informatif.

5.2 Reproductibilité et synthèse

L'ensemble des expériences a été conduit avec des graines aléatoires fixées (découpage des plis, ré-échantillonnage bootstrap, initialisation des solveurs), garantissant la reproductibilité exacte des chiffres rapportés. Le test de permutation ($p = 0,002$, sous-section précédente) confirme par ailleurs que le signal prédictif n'est pas un artefact du hasard.

En synthèse, nous distinguons :

- **Conclusions robustes** : le signal discriminant est porté par le bloc d'expression génique (GE) ; le bloc CGH est non informatif ; les classes `cort` et `dipg` sont bien séparées ; et le signal prédictif global est statistiquement significatif.
- **Conclusions fragiles** : le classement fin des deux ou trois meilleures méthodes, le recall exact

sur la classe `midl` (qui se joue sur 1 à 2 patients), et la liste précise des gènes sélectionnés, qui varie d'un ré-échantillon à l'autre autour d'un noyau stable.

Pour garantir que les performances de nos modèles ne résultent pas d'un simple artefact statistique, nous avons réalisé un test de permutation [OG10]. Ce test non paramétrique permet d'évaluer la robustesse du lien entre les données moléculaires et la localisation tumorale.

Principe et Hypothèses

L'objectif est de comparer la performance observée de nos classifieurs à une distribution de performances obtenues sous l'hypothèse nulle.

- Hypothèse nulle H_0 : Il n'existe aucune relation de dépendance entre les variables et les étiquettes de classes. Sous cette hypothèse, la balanced accuracy moyenne attendue est de 1/3 pour notre problème à trois classes .
- Hypothèse alternative H_1 : Le modèle a appris un signal biologique réel permettant de discriminer les localisations.

Procédure algorithmique

Le test suit les étapes :

- Désaccouplement : lien entre les observations X et les labels Y brisés par des permutations aléatoires.
- La pipeline retourne entièrement et on ré-entraine un modèle sur les données permutées
- On répète l'opération $B = 500$ fois pour construire une distribution empirique des scores sous H_0
- Calcul de la p-value par la proportion de permutations ayant donné un score supérieur ou égal au score observé sur les données réelles.

Réalisation et Résultats : Tous les modèles obtiennent une **p-value de** $0,002$. Nous pouvons donc rejeter H_0 avec une grande confiance et affirmer que le signal prédictif identifié est statistiquement significatif et non dû au hasard .

Références

[Cha+02] Nitesh V. Chawla et al. “SMOTE: synthetic minority over-sampling technique”. In: *Journal of Artificial Intelligence Research* 16 (2002), pp. 321–357. DOI: 10.1613/jair.953.

[Din+22] Daisy Yi Ding et al. “Cooperative learning for multiview analysis”. In: *Proceedings of the National Academy of Sciences* 119.38 (2022), e2202113119. DOI: 10.1073/pnas.2202113119.

[Efr79] Bradley Efron. “Bootstrap methods: another look at the jackknife”. In: *The Annals of Statistics* 7.1 (1979), pp. 1–26.

[FHT10] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. “Regularization paths for generalized linear models via coordinate descent”. In: *Journal of Statistical Software* 33.1 (2010), pp. 1–22. DOI: 10.18637/jss.v033.i01.

[MB10] Nicolai Meinshausen and Peter Bühlmann. “Stability selection”. In: *Journal of the Royal Statistical Society: Series B* 72.4 (2010), pp. 417–473. DOI: 10.1111/j.1467-9868.2010.00740.x.

[OG10] Markus Ojala and Gemma C. Garriga. “Permutation tests for studying classifier performance”. In: *Journal of Machine Learning Research* 11 (2010), pp. 1833–1863.

[Ten+14] Arthur Tenenhaus et al. “Variable selection for generalized canonical correlation analysis”. In: *Biostatistics* 15.3 (2014), pp. 569–583. DOI: 10.1093/biostatistics/kxu001.

[Tib+05] Robert Tibshirani et al. “Sparsity and smoothness via the fused lasso”. In: *Journal of the Royal Statistical Society: Series B* 67.1 (2005), pp. 91–108. DOI: 10.1111/j.1467-9868.2005.00490.x.

[Tib96] Robert Tibshirani. “Regression shrinkage and selection via the lasso”. In: *Journal of the Royal Statistical Society: Series B* 58.1 (1996), pp. 267–288. DOI: 10.1111/j.2517-6161.1996.tb02080.x.

[TT11] Arthur Tenenhaus and Michel Tenenhaus. “Regularized Generalized Canonical Correlation Analysis”. In: *Psychometrika* 76.2 (2011), pp. 257–284. DOI: 10.1007/s11336-011-9206-8.

[ZH05] Hui Zou and Trevor Hastie. “Regularization and variable selection via the elastic net”. In: *Journal of the Royal Statistical Society: Series B* 67.2 (2005), pp. 301–320. DOI: 10.1111/j.1467-9868.2005.00503.x.